

Separating Logic and Compositional Symbolic Execution

Lecture 4:
Semi-automatic Verification & Automatic Bug Detection

Link:

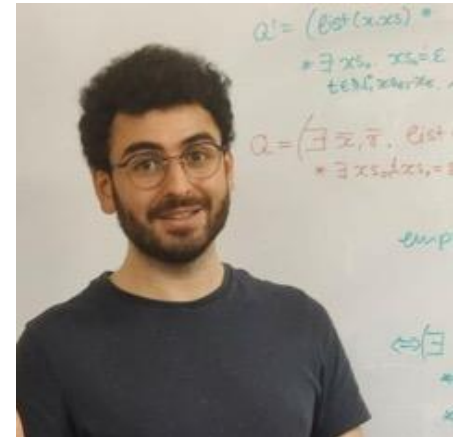
<https://gillianplatform.github.io/labs/oplss26/>



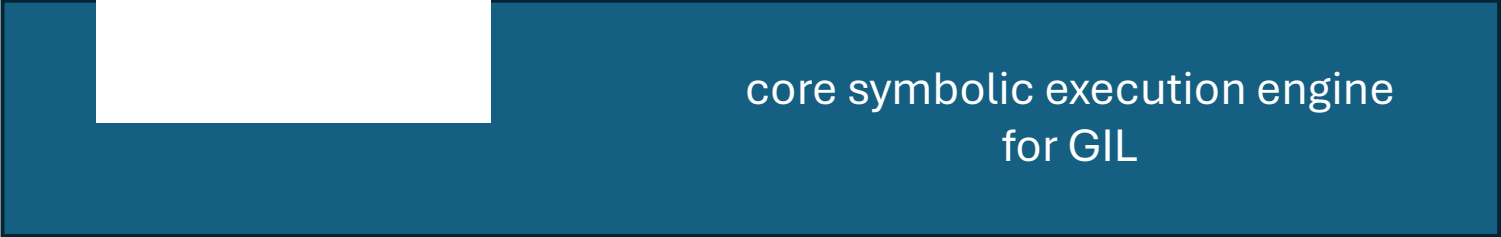
Gillian Platform: PLDI'20, CAV'21

A general platform for the development of compositional analysis tools with strong mathematical foundations:

- Parametric on the partial symbolic state
- Gillian-specific match/consume and produce algorithms satisfy the properties of the algebraic interface
- Gillian-JavaScript and Gillian-C and Gillian-Rust [PLDI'25] tool instantiations
- Applications to semi-automatic verification and true bug detection for real-world code:
e.g., verification of the JS implementation (200 loc) and C implementation (900 loc) of critical AWS encryption SDK code



Gillian: Parametric Symbolic Execution

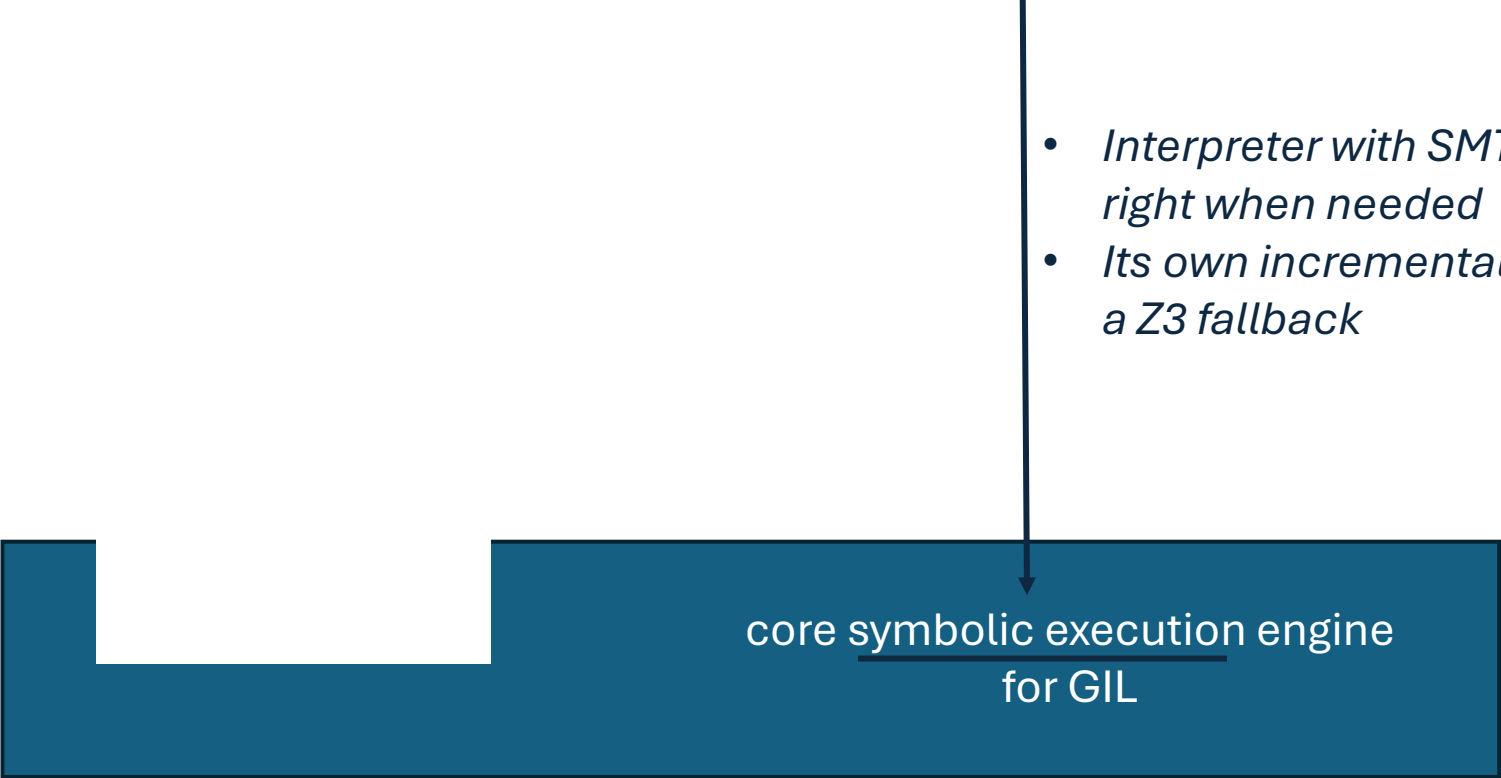


core symbolic execution engine
for GIL

Gillian: Parametric Symbolic Execution

quite literally speaking

- *Interpreter with SMT queries sent right when needed*
- *Its own incremental SMT solver with a Z3 fallback*



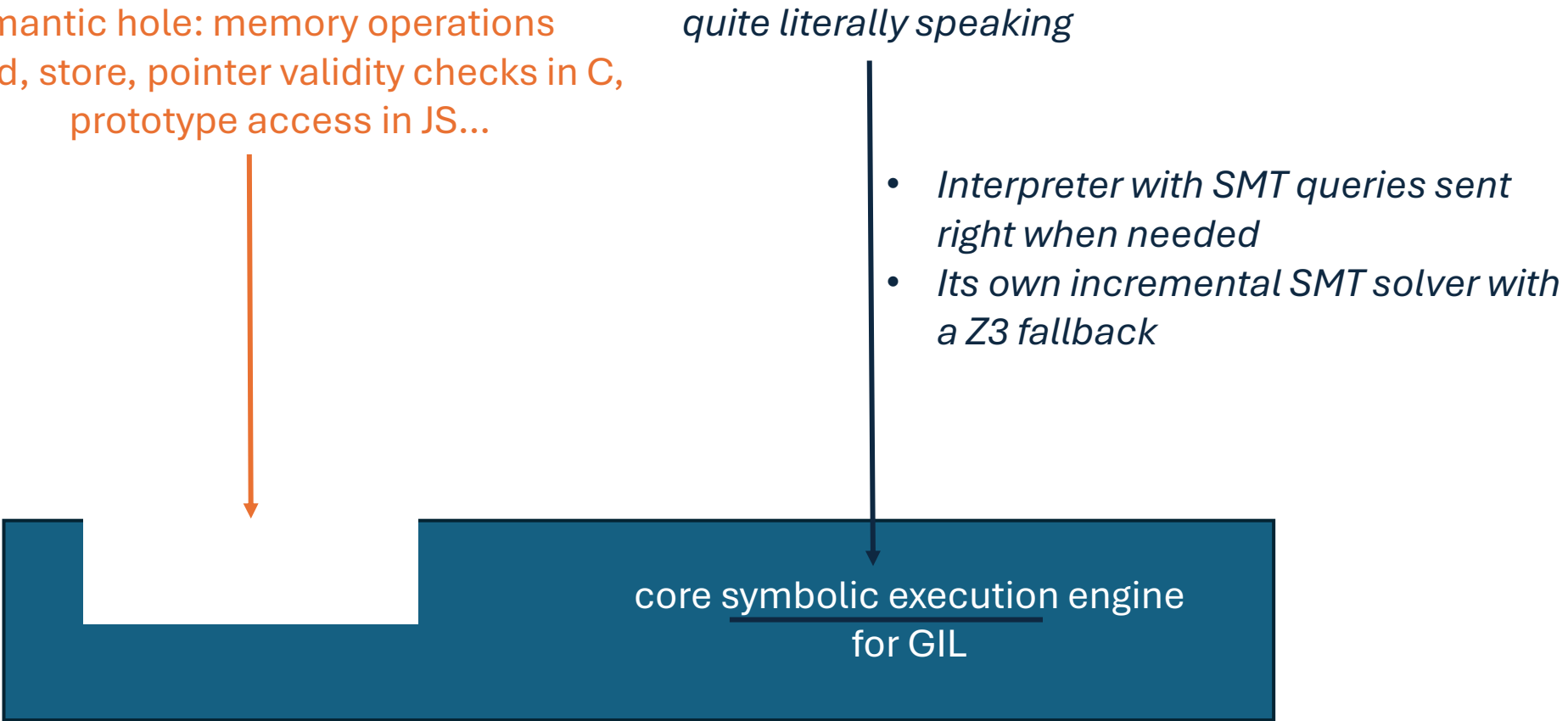
core symbolic execution engine
for GIL

Gillian: Parametric Symbolic Execution

Semantic hole: memory operations
e.g. load, store, pointer validity checks in C,
prototype access in JS...

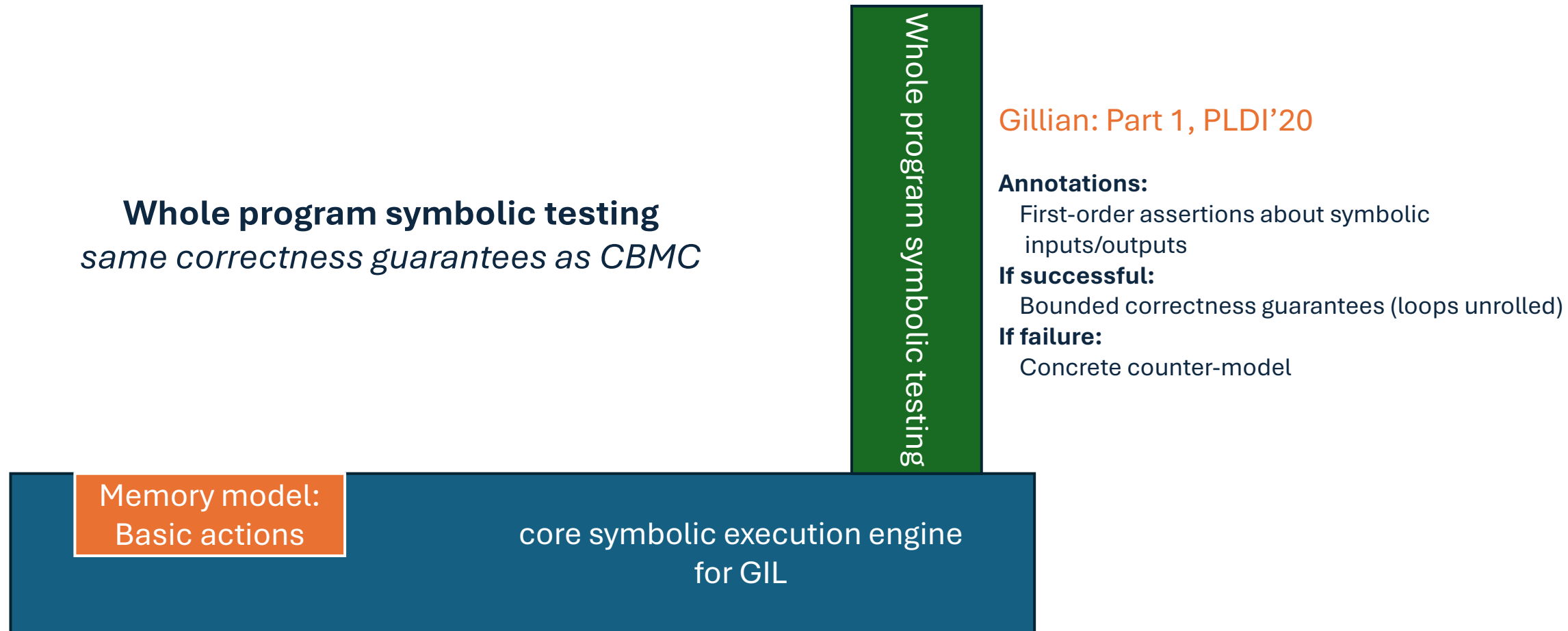
quite literally speaking

- *Interpreter with SMT queries sent right when needed*
- *Its own incremental SMT solver with a Z3 fallback*



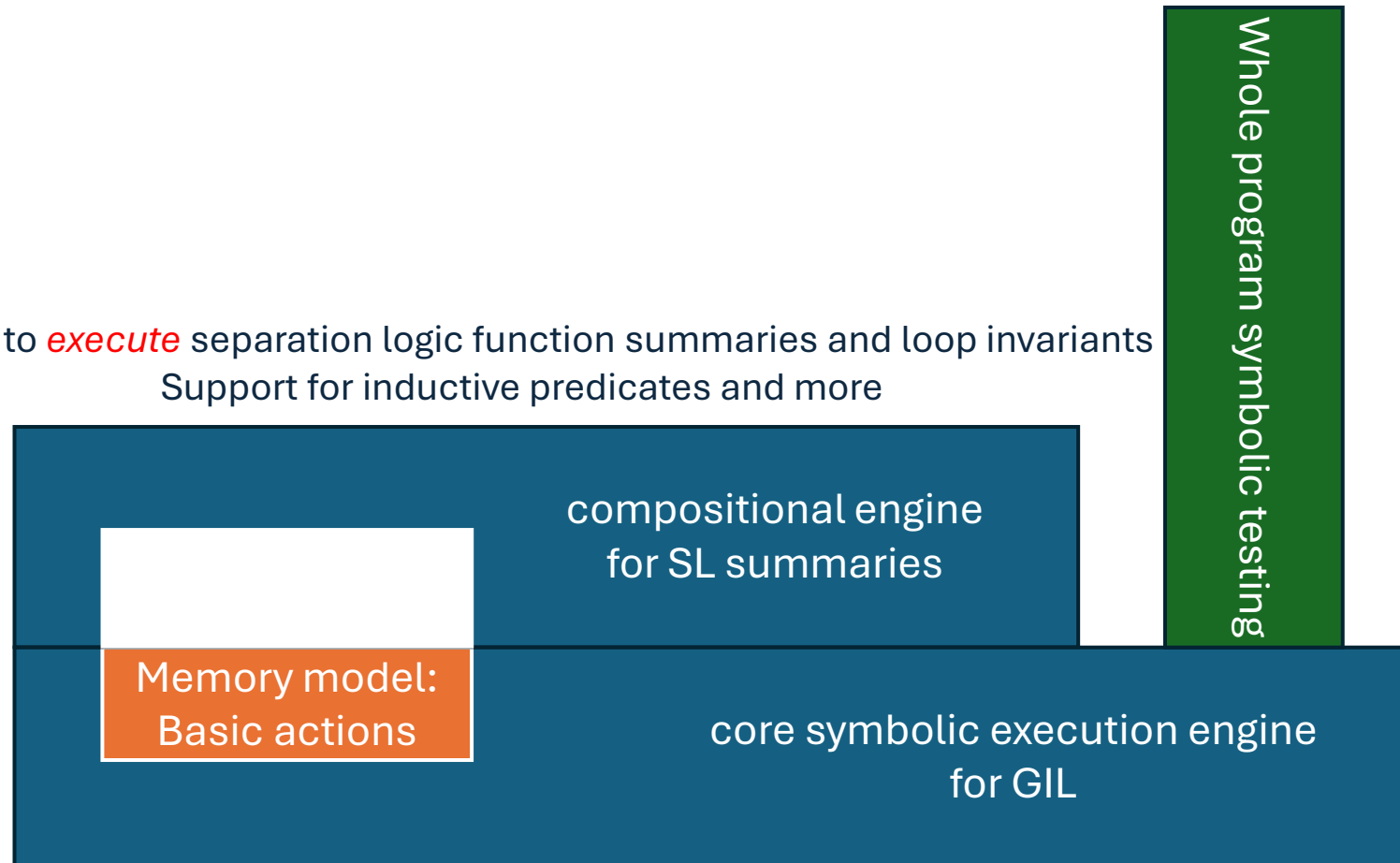
core symbolic execution engine
for GIL

Gillian: Parametric Symbolic Execution



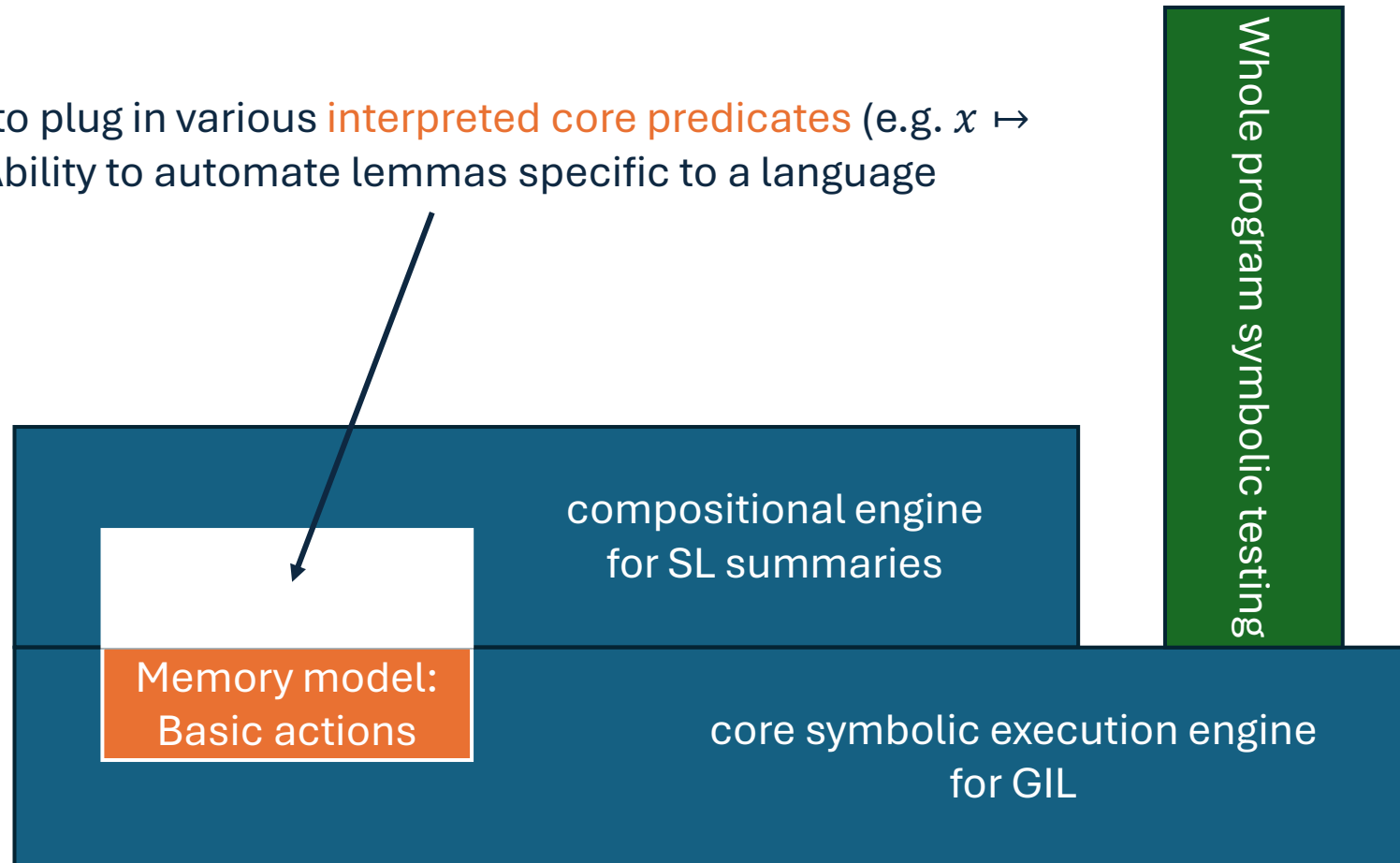
Gillian: Compositional Parametric Symbolic Execution

Ability to *execute* separation logic function summaries and loop invariants
Support for inductive predicates and more



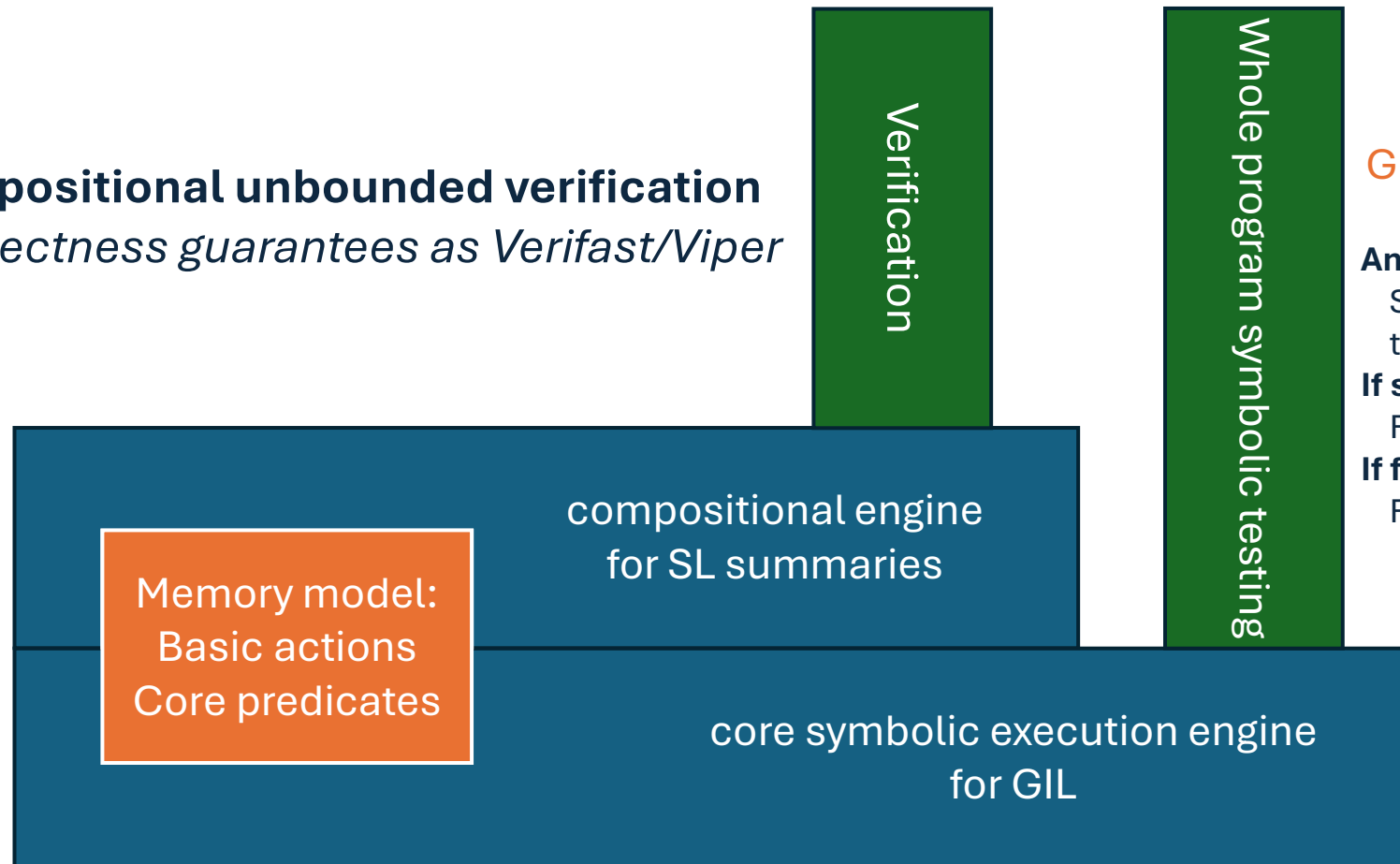
Gillian: Compositional Parametric Symbolic Execution

Ability to plug in various **interpreted core predicates** (e.g. $x \mapsto y$) => Ability to automate lemmas specific to a language



Gillian: Compositional Parametric Symbolic Execution

Full compositional unbounded verification
same correctness guarantees as Verifast/Viper



Gillian: Part 2, CAV'21

Annotations:

Separation logic specs, invariants, lemmas, tactics...

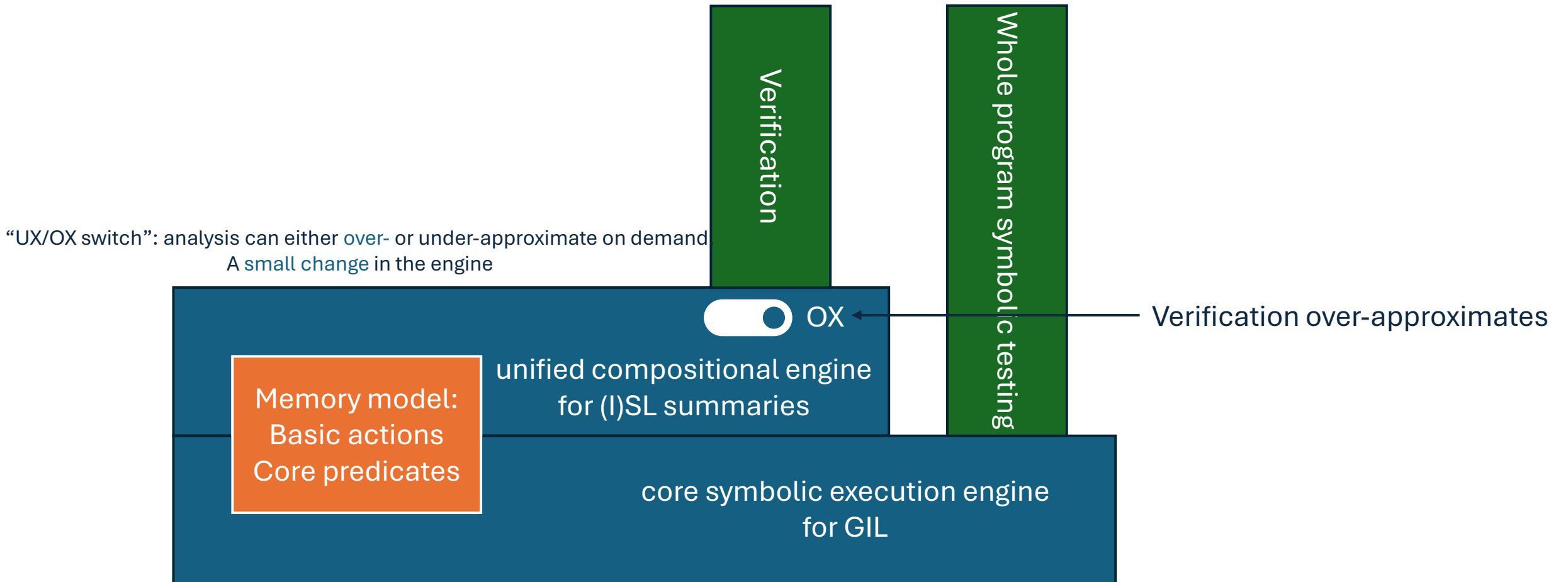
If successful:

Functional correctness

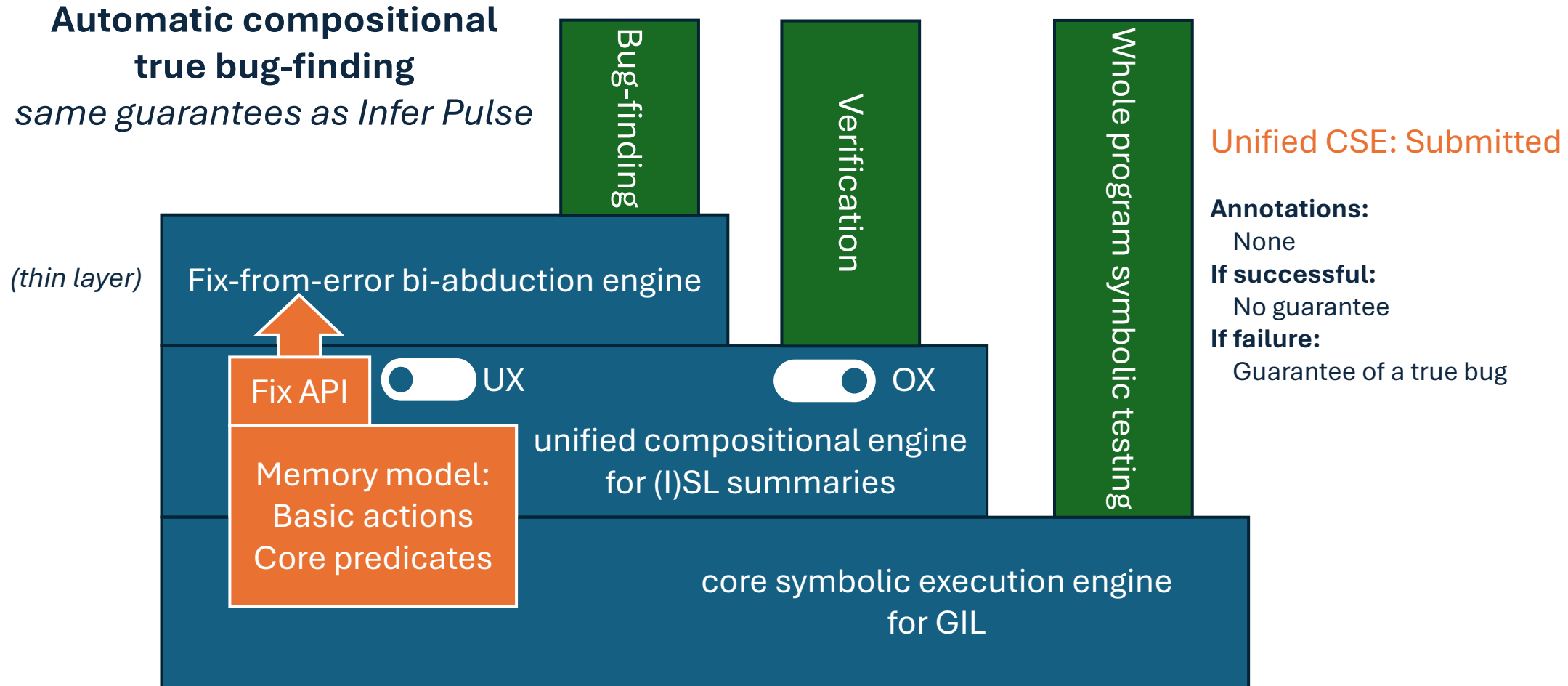
If failure:

Failing symbolic trace

Gillian: Unified Compositional Parametric Symbolic Execution



Gillian: Unified Compositional Parametric Symbolic Execution



Gillian Instantiations

WiSL (Gillian-While)	Simple block-offset model for teaching and experimentation Applied to data-structure algorithms
Gillian-JS	Extensible object memory model, JaVerT compiler Used for symbolic testing of Buckets-JS & verification of code from AWS Encryption SDK
Gillian-C	Block-offset memory model, CompCert and CBMC compilers Used for symbolic testing of Collections-C & verification of code from AWS Encryption SDK
Gillian-Rust	Step up from Gillian-C; partially laid-out heap, Iris ghost resources; Rust compiler + ours Used to verify parts of standard library (LinkedList & Vec)
Gillian-CHERI	Extension of Gillian-C memory model with support for CHERI capabilities

Gillian Student Lab



Gillian Verification Debugging

```

769 PFS/Gamma simplification:
770 With unification: No
771 PFS:
772 (w_el = { { #loc1, #offset1 } })
773 (w_er = { { #locr, #offsetr } })
774 (w_lor = #loc1)
775 (w_out = (offset1 < #offsetr))
776
777 Gamma:
778
779
780 Analysing list structure.
781 Analysing list structure.
782 PFS/Gamma simplification completed:
783
784 PFS:
785
786
787 Gamma:
788 (w_offset1: Int)
789 (w_offsetr: Int)
790
791 PFS/Gamma simplification:
792 With unification: No
793 PFS:
794 (w_el = { { #loc1, #offset1 } })
795 (w_er = { { #locr, #offsetr } })
796 (w_lor = #loc1)
797 (w_out = (offset1 < #offsetr))
798
799 Gamma:
800 (w_el: List)
801 (w_er: List)
802
803 Analysing list structure.
804 Analysing list structure.
805 PFS/Gamma simplification completed:
806
807 PFS:
808

```

```

3054 list(x, walpha)
3055
3056
3057
3058
3059
3060
3061
3062 Strategy 1: Examining list(x, walpha)
3063 Original values: [x, walpha]
3064 Extended values: [null, x]
3065 get_pred_with_vs. Strategy 1. Intersection of cardinal 1: [x]
3066 FOUND STH TO UNFOLD: list!!!!
3067
3068 Combine going to explode. PredPhase: list. Params: x, walpha. Args: x, walpha
3069
3070
3071
3072 Using unfold info, obtained subst:
3073 [ (alpha: #alpha), (x: x) ]
3074
3075 Going to produce 2 definitions with subst
3076 [ (alpha: #alpha), (x: x) ]
3077
3078
3079 Produce assertion: types(alpha : List, wbeta : List, wval1_0 : Obj,
3080
3081
3082
3083
3084
3085
3086
3087 Produce simple assertion: types(alpha : List)
3088 With subst: [ (alpha: #alpha), (x: x) ]
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128

```

```

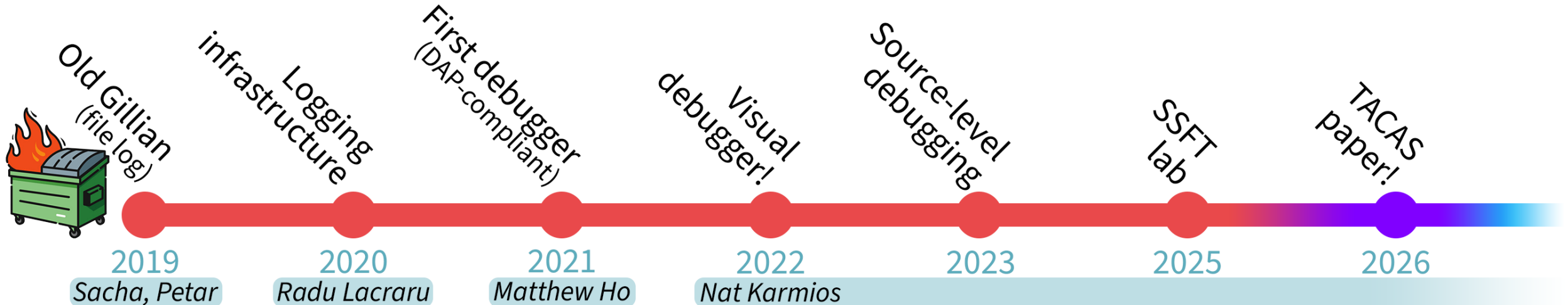
5000 TYPING ENVIRONMENT:
5001 (alpha: List)
5002 (w: List)
5003 (_lvar_5: List)
5004 (_lvar_6: Obj)
5005 (_lvar_7: Int)
5006
5007 PFS/Gamma simplification:
5008 With unification: No
5009 PFS:
5010 (w = { { _l1, _lvar_7 } })
5011 (walpha = { { (_lvar_6), _lvar_5 } })
5012 (_lvar_6 = _l1)
5013 (01 <= (1-len walpha))
5014 (01 <= (1-len x))
5015 (01 <= (1-len _lvar_5))
5016
5017 Gamma:
5018 (walpha: List)
5019 (w: List)
5020 (_lvar_5: List)
5021 (_lvar_6: Obj)
5022 (_lvar_7: Int)
5023
5024 Analysing list structure.
5025 PFS/Gamma simplification completed:
5026
5027 PFS:
5028 (w = { { _l1, _lvar_7 } })
5029 (walpha = { { (_lvar_6), _lvar_5 } })
5030 (_lvar_6 = _l1)
5031 (01 <= (1-len walpha))
5032 (01 <= (1-len x))
5033 (01 <= (1-len _lvar_5))
5034
5035 Gamma:
5036 (walpha: List)
5037 (w: List)
5038 (_lvar_5: List)
5039 (_lvar_6: Obj)
5040 (_lvar_7: Int)
5041

```

```

7783 Unify UP! There are 1 states left to consider.
7784 Substs:
7785 []
7786 [walpha]
7787 []
7788
7789 Unify assertion: (ret = (1-len walpha)),
7790 Subst:
7791 [ (ret: 01), (walpha: walpha), (w: w) ]
7792
7793 STATE:
7794 SPEC VARS: walpha, w
7795 STORE:
7796 (ret: List 01)
7797
7798 MEMORY:
7799
7800 PURE FORMULAE:
7801 (w = null)
7802 (walpha = { { } })
7803 (01 <= (1-len walpha))
7804
7805 TYPING ENVIRONMENT:
7806 (walpha: List)
7807 (w: Null)
7808
7809 PREDS:
7810
7811
7812
7813
7814
7815
7816
7817
7818
7819
7820
7821
7822
7823
7824
7825
7826
7827
7828
7829
7830
7831
7832
7833
7834
7835
7836
7837
7838
7839
7840
7841
7842
7843
7844
7845
7846
7847
7848
7849
7850
7851
7852
7853
7854
7855
7856
7857
7858
7859
7860
7861
7862
7863
7864
7865
7866
7867
7868
7869
7870
7871
7872
7873
7874
7875
7876
7877
7878
7879
7880
7881
7882
7883
7884
7885
7886
7887
7888
7889
7890
7891
7892
7893
7894
7895
7896
7897
7898
7899
7900
7901
7902
7903
7904
7905
7906
7907
7908
7909
7910
7911
7912
7913
7914
7915
7916
7917
7918
7919
7920
7921
7922
7923
7924
7925
7926
7927
7928
7929
7930
7931
7932
7933
7934
7935
7936
7937
7938
7939
7940
7941
7942
7943
7944
7945
7946
7947
7948
7949
7950
7951
7952
7953
7954
7955
7956
7957
7958
7959
7960
7961
7962
7963
7964
7965
7966
7967
7968
7969
7970
7971
7972
7973
7974
7975
7976
7977
7978
7979
7980
7981
7982
7983
7984
7985
7986
7987
7988
7989
7990
7991
7992
7993
7994
7995
7996
7997
7998
7999
8000
8001
8002
8003
8004
8005
8006
8007
8008
8009
8010
8011
8012
8013
8014
8015
8016
8017
8018
8019
8020
8021
8022
8023
8024
8025
8026
8027
8028
8029
8030
8031
8032
8033
8034
8035
8036
8037
8038
8039
8040
8041
8042
8043
8044
8045
8046
8047
8048
8049
8050
8051
8052
8053
8054
8055
8056
8057
8058
8059
8060
8061
8062
8063
8064
8065
8066
8067
8068
8069
8070
8071
8072
8073
8074
8075
8076
8077
8078
8079
8080
8081
8082
8083
8084
8085
8086
8087
8088
8089
8090
8091
8092
8093
8094
8095
8096
8097
8098
8099
8100
8101
8102
8103
8104
8105
8106
8107
8108
8109
8110
8111
8112
8113
8114
8115
8116
8117
8118
8119
8120
8121
8122
8123
8124
8125
8126
8127
8128
8129
8130
8131
8132
8133
8134
8135
8136
8137
8138
8139
8140
8141
8142
8143
8144
8145
8146
8147
8148
8149
8150
8151
8152
8153
8154
8155
8156
8157
8158
8159
8160
8161
8162
8163
8164
8165
8166
8167
8168
8169
8170
8171
8172
8173
8174
8175
8176
8177
8178
8179
8180
8181
8182
8183
8184
8185
8186
8187
8188
8189
8190
8191
8192
8193
8194
8195
8196
8197
8198
8199
8200
8201
8202
8203
8204
8205
8206
8207
8208
8209
8210
8211
8212
8213
8214
8215
8216
8217
8218
8219
8220
8221
8222
8223
8224
8225
8226
8227
8228
8229
8230
8231
8232
8233
8234
8235
8236
8237
8238
8239
8240
8241
8242
8243
8244
8245
8246
8247
8248
8249
8250
8251
8252
8253
8254
8255
8256
8257
8258
8259
8260
8261
8262
8263
8264
8265
8266
8267
8268
8269
8270
8271
8272
8273
8274
8275
8276
8277
8278
8279
8280
8281
8282
8283
8284
8285
8286
8287
8288
8289
8290
8291
8292
8293
8294
8295
8296
8297
8298
8299
8300
8301
8302
8303
8304
8305
8306
8307
8308
8309
8310
8311
8312
8313
8314
8315
8316
8317
8318
8319
8320
8321
8322
8323
8324
8325
8326
8327
8328
8329
8330
8331
8332
8333
8334
8335
8336
8337
8338
8339
8340
8341
8342
8343
8344
8345
8346
8347
8348
8349
8350
8351
8352
8353
8354
8355
8356
8357
8358
8359
8360
8361
8362
8363
8364
8365
8366
8367
8368
8369
8370
8371
8372
8373
8374
8375
8376
8377
8378
8379
8380
8381
8382
8383
8384
8385
8386
8387
8388
8389
8390
8391
8392
8393
8394
8395
8396
8397
8398
8399
8400
8401
8402
8403
8404
8405
8406
8407
8408
8409
8410
8411
8412
8413
8414
8415
8416
8417
8418
8419
8420
8421
8422
8423
8424
8425
8426
8427
8428
8429
8430
8431
8432
8433
8434
8435
8436
8437
8438
8439
8440
8441
8442
8443
8444
8445
8446
8447
8448
8449
8450
8451
8452
8453
8454
8455
8456
8457
8458
8459
8460
8461
8462
8463
8464
8465
8466
8467
8468
8469
8470
8471
8472
8473
8474
8475
8476
8477
8478
8479
8480
8481
8482
8483
8484
8485
8486
8487
8488
8489
8490
8491
8492
8493
8494
8495
8496
8497
8498
8499
8500
8501
8502
8503
8504
8505
8506
8507
8508
8509
8510
8511
8512
8513
8514
8515
8516
8517
8518
8519
8520
8521
8522
8523
8524
8525
8526
8527
8528
8529
8530
8531
8532
8533
8534
8535
8536
8537
8538
8539
8540
8541
8542
8543
8544
8545
8546
8547
8548
8549
8550
8551
8552
8553
8554
8555
8556
8557
8558
8559
8560
8561
8562
8563
8564
8565
8566
8567
8568
8569
8570
8571
8572
8573
8574
8575
8576
8577
8578
8579
8580
8581
8582
8583
8584
8585
8586
8587
8588
8589
8590
8591
8592
8593
8594
8595
8596
8597
8598
8599
8600
8601
8602
8603
8604
8605
8606
8607
8608
8609
8610
8611
8612
8613
8614
8615
8616
8617
8618
8619
8620
8621
8622
8623
8624
8625
8626
8627
8628
8629
8630
8631
8632
8633
8634
8635
8636
8637
8638
8639
8640
8641
8642
8643
8644
8645
8646
8647
8648
8649
8650
8651
8652
8653
8654
8655
8656
8657
8658
8659
8660
8661
8662
8663
8664
8665
8666
8667
8668
8669
8670
8671
8672
8673
8674
8675
8676
8677
8678
8679
8680
8681
8682
8683
8684
8685
8686
8687
8688
8689
8690
8691
8692
8693
8694
8695
8696
8697
8698
8699
8700
8701
8702
8703
8704
8705
8706
8707
8708
8709
8710
8711
8712
8713
8714
8715
8716
8717
8718
8719
8720
8721
8722
8723
8724
8725
8726
8727
8728
8729
8730
8731
8732
8733
8734
8735
8736
8737
8738
8739
8740
8741
8742
8743
8744
8745
8746
8747
8748
8749
8750
8751
8752
8753
8754
8755
8756
8757
8758
8759
8760
8761
8762
8763
8764
8765
8766
8767
8768
8769
8770
8771
8772
8773
8774
8775
8776
8777
8778
8779
8780
8781
8782
8783
8784
8785
8786
8787
8788
8789
8790
8791
8792
8793
8794
8795
8796
8797
8798
8799
8800
8801
8802
8803
8804
8805
8806
8807
8808
8809
8810
8811
8812
8813
8814
8815
8816
8817
8818
8819
8820
8821
8822
8823
8824
8825
8826
8827
8828
8829
8830
8831
8832
8833
8834
8835
8836
8837
8838
8839
8840
8841
8842
8843
8844
8845
8846
8847
8848
8849
8850
8851
8852
8853
8854
8855
8856
8857
8858
8859
8860
8861
8862
8863
8864
8865
8866
8867
8868
8869
8870
8871
8872
8873
8874
8875
8876
8877
8878
8879
8880
8881
8882
8883
8884
8885
8886
8887
8888
8889
8890
8891
8892
8893
8894
8895
8896
8897
8898
8899
8900
8901
8902
8903
8904
8905
8906
8907
8908
8909
8910
8911
8912
8913
8914
8915
8916
8917
8918
8919
8920
8921
8922
8923
8924
8925
8926
8927
8928
8929
8930
8931
8932
8933
8934
8935
8936
8937
8938
8939
8940
8941
8942
8943
8944
8945
8946
8947
8948
8949
8950
8951
8952
8953
8954
8955
8956
8957
8958
8959
8960
8961
8962
8963
8964
8965
8966
8967
8968
8969
8970
8971
8972
8973
8974
8975
8976
8977
8978
8979
8980
8981
8982
8983
8984
8985
8986
8987
8988
8989
8990
8991
8992
8993
8994
8995
8996
8997
8998
8999
9000

```



WiSL demo

Gillian-While with block-offset memory

https://youtu.be/31LeZB_91L4



Diego Cupello

Our C Memory Model

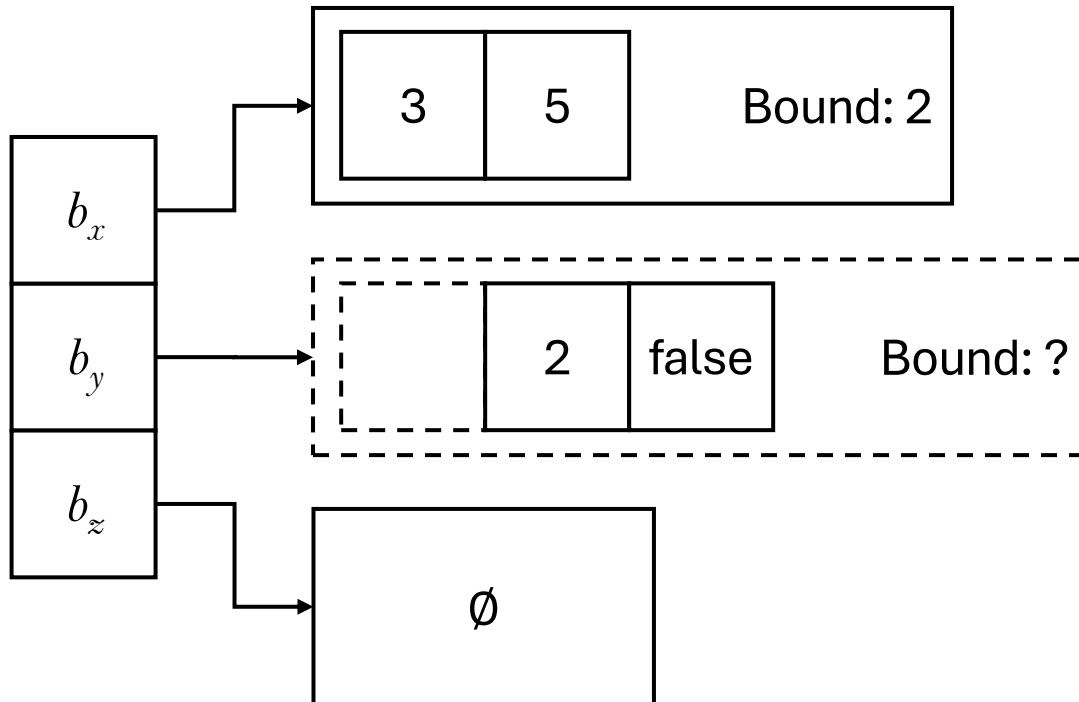
- Based on the CompCert C Memory Model
- Extension of the simple block-offset memory model



Block-offset Model (recap)

Partial Memory

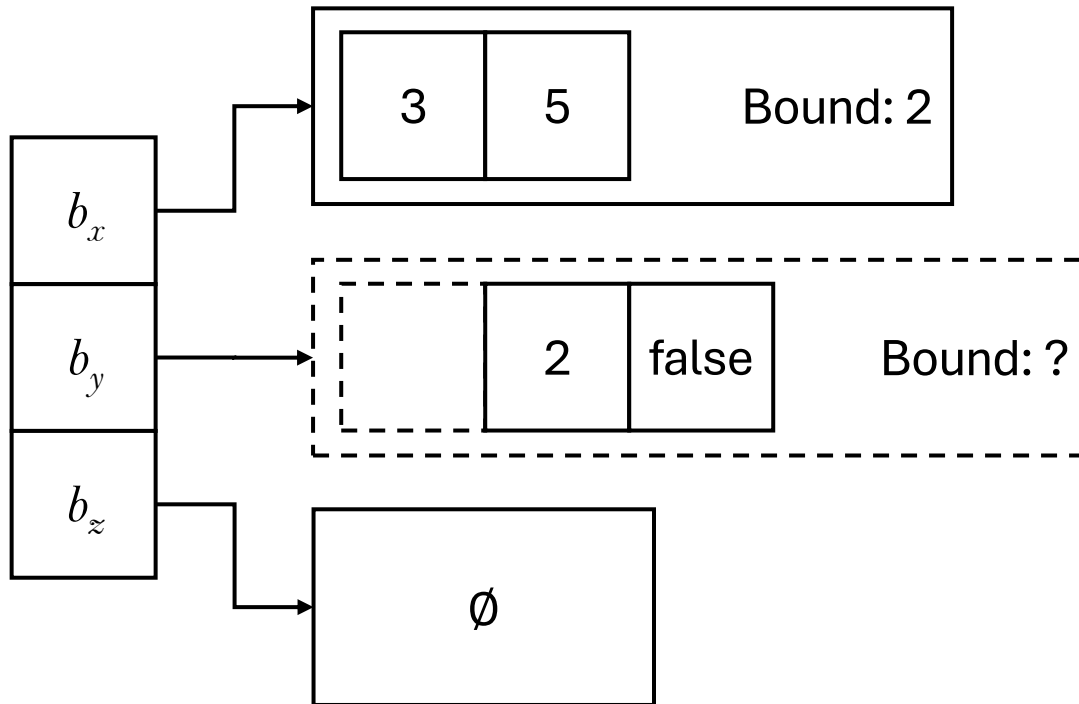
$$h : \text{Loc} \rightarrow_{\text{fin}} ((\mathbb{N} \rightarrow_{\text{fin}} \text{Val}) \times \mathbb{N}^?)_{\emptyset}$$



Block-offset Model (recap)

Memory Actions

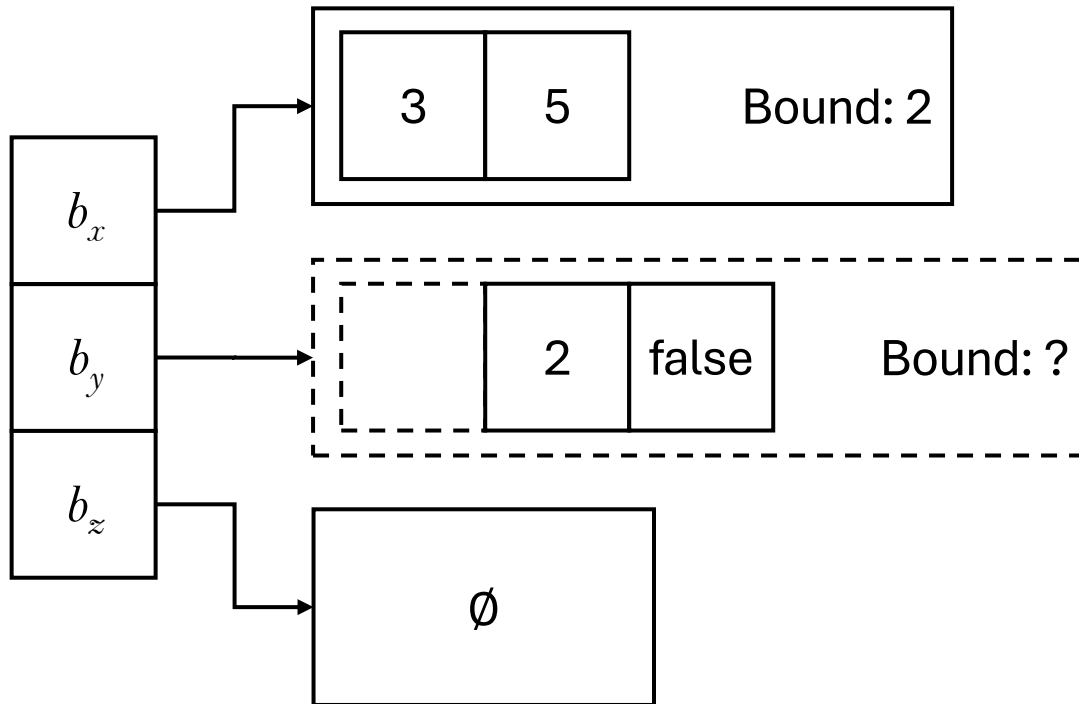
$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$



Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$

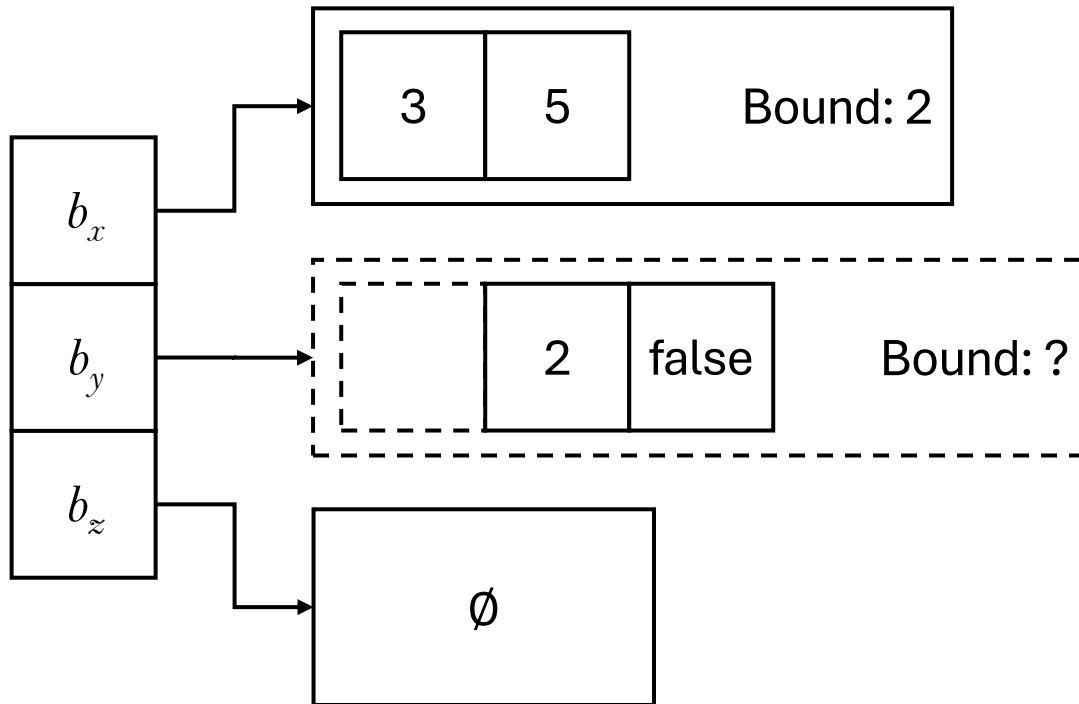


$\text{load}(b_x, 1)$ returns 5

Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$



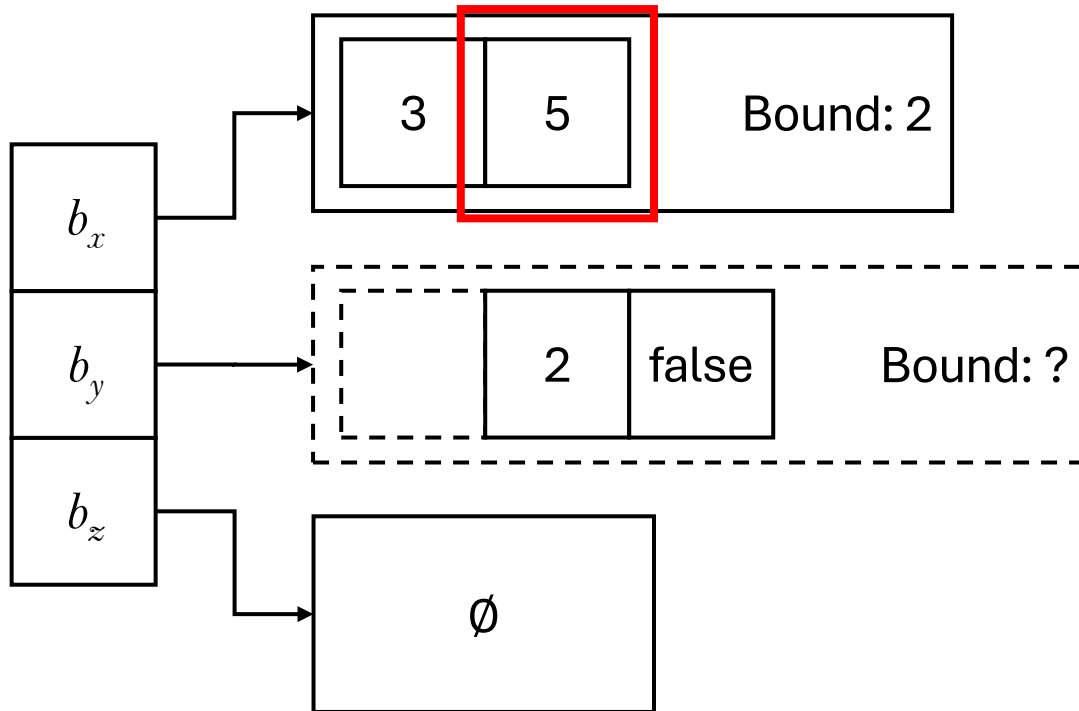
$\text{load}(b_x, 1)$ returns 5

Load value at location b_x , offset 1

Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$



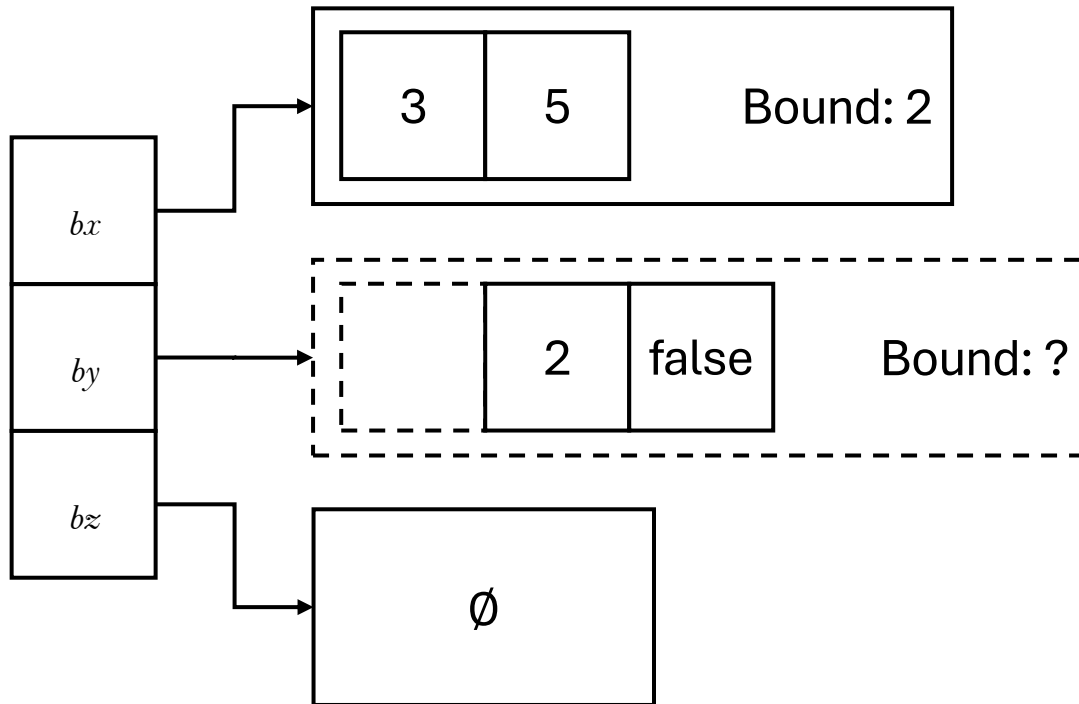
$\text{load}(b_x, 1)$ returns 5

Load value at location b_x , offset 1

Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$

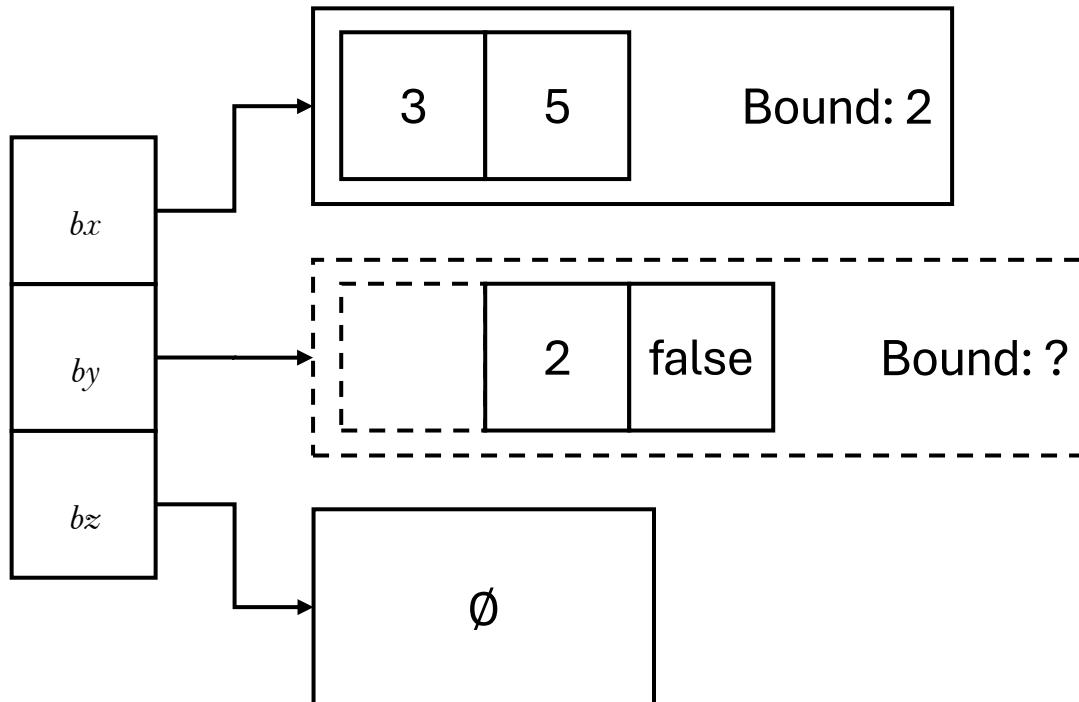


$\text{store}(b_x, 0, 9)$

Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$



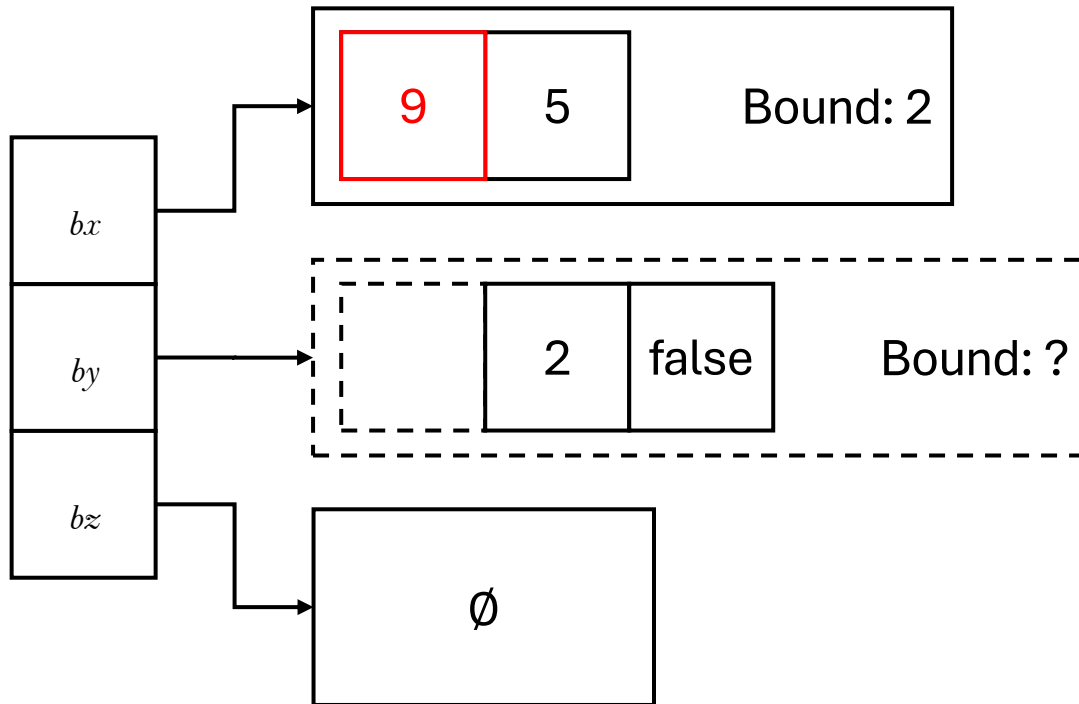
$\text{store}(b_x, 0, 9)$

Store value 9 at block b_x , offset 0

Block-offset Model (recap)

Memory Actions

$\text{new}(E) \mid \text{free}(E) \mid \text{load}(E, E) \mid \text{store}(E, E, E)$



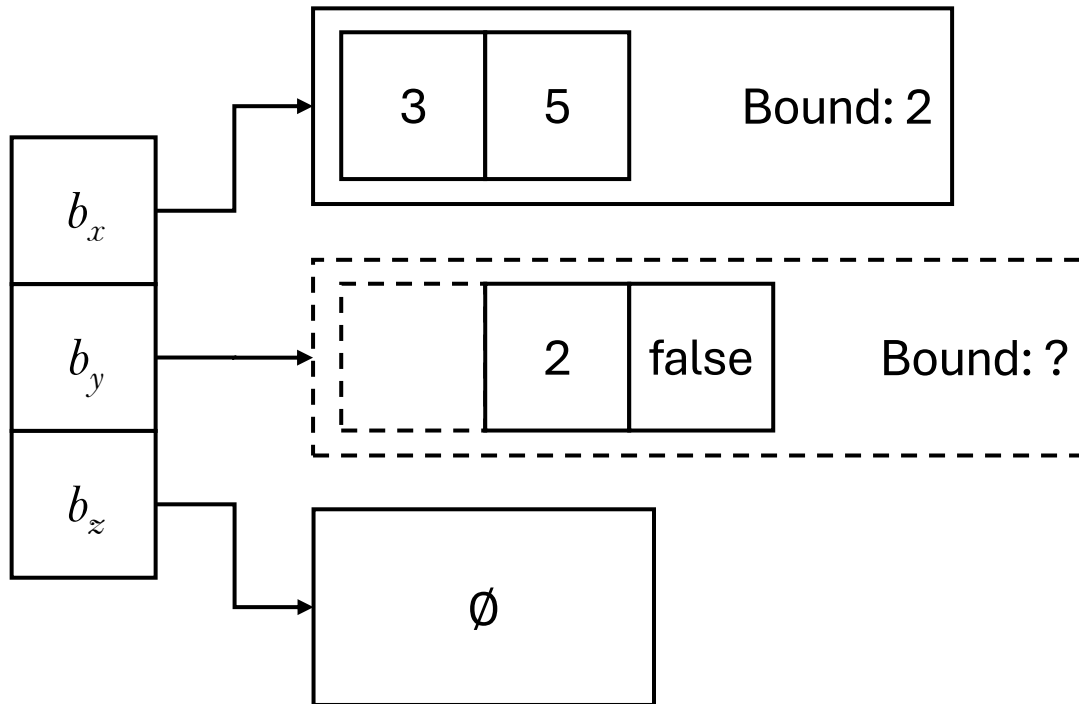
$\text{store}(b_x, 0, 9)$

Store value 9 at block b_x , offset 0

Block-offset Model (recap)

Resource Assertions

$$E \mapsto E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

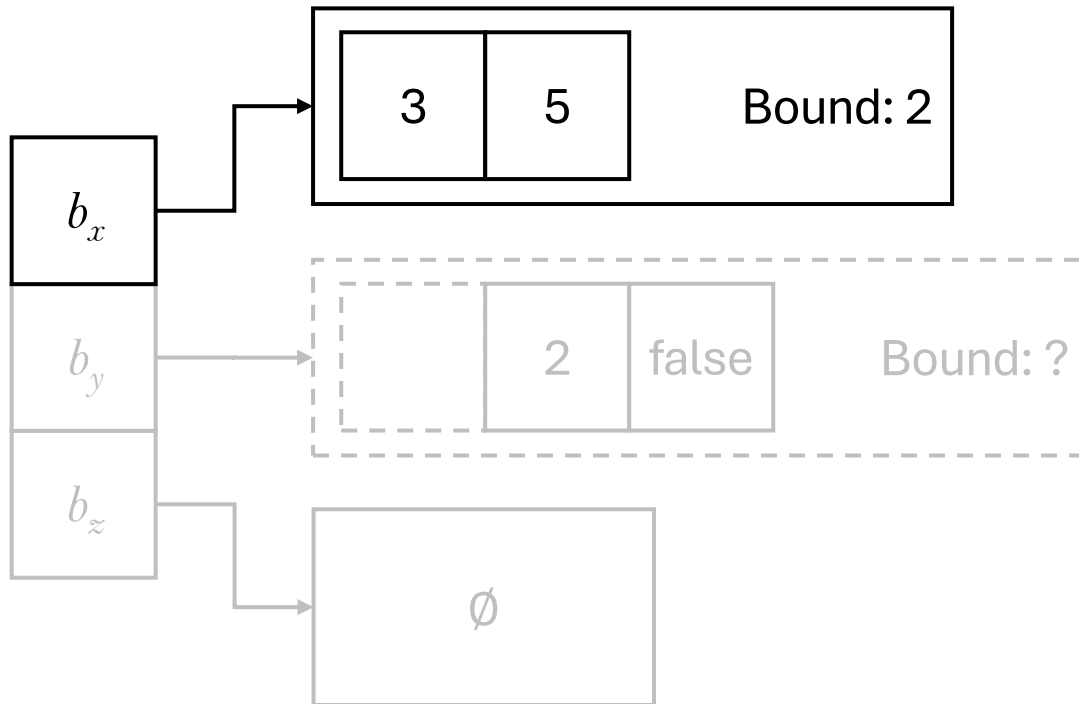


$$b_x \mapsto^b 3, 5 \quad \star \quad b_y + 1 \mapsto 2, \text{false} \quad \star \quad b_z \mapsto \emptyset$$

Block-offset Model (recap)

Resource Assertions

$$E \mapsto E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$



$$b_x \mapsto^b 3, 5$$

★

$$b_y + 1 \mapsto 2, \text{false}$$

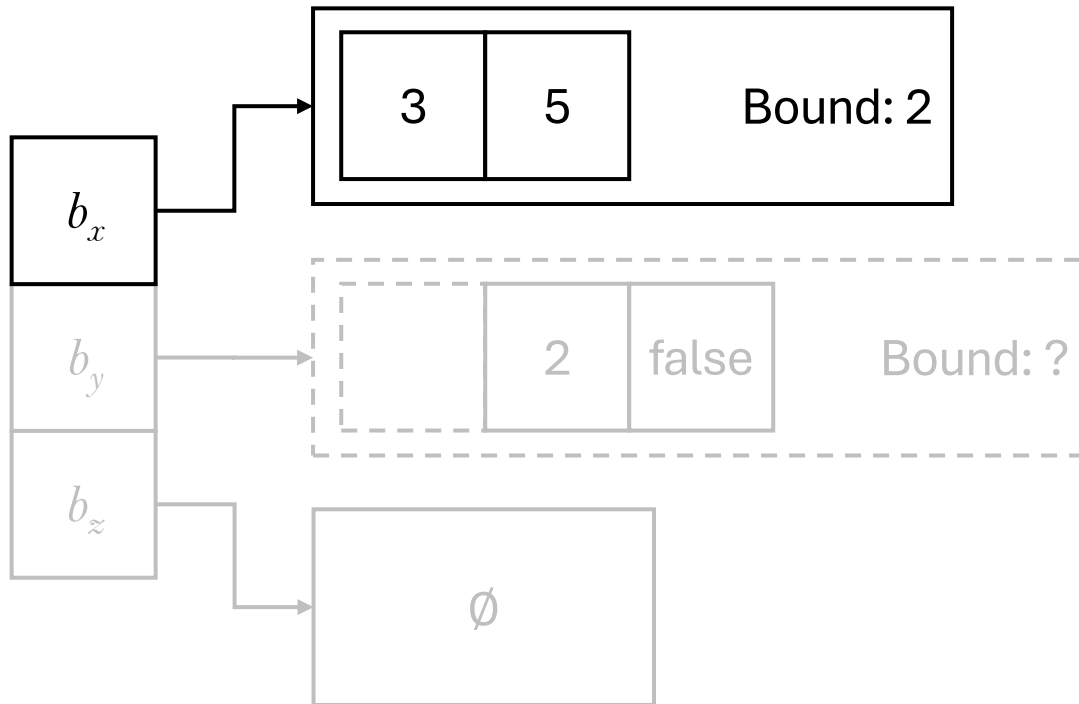
★

$$b_z \mapsto \emptyset$$

Block-offset Model (recap)

Resource Assertions

$$E \mapsto E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$



$$b_x \mapsto^b 3, 5 \quad \star \quad b_y + 1 \mapsto 2, \text{false} \quad \star \quad b_z \mapsto \emptyset$$

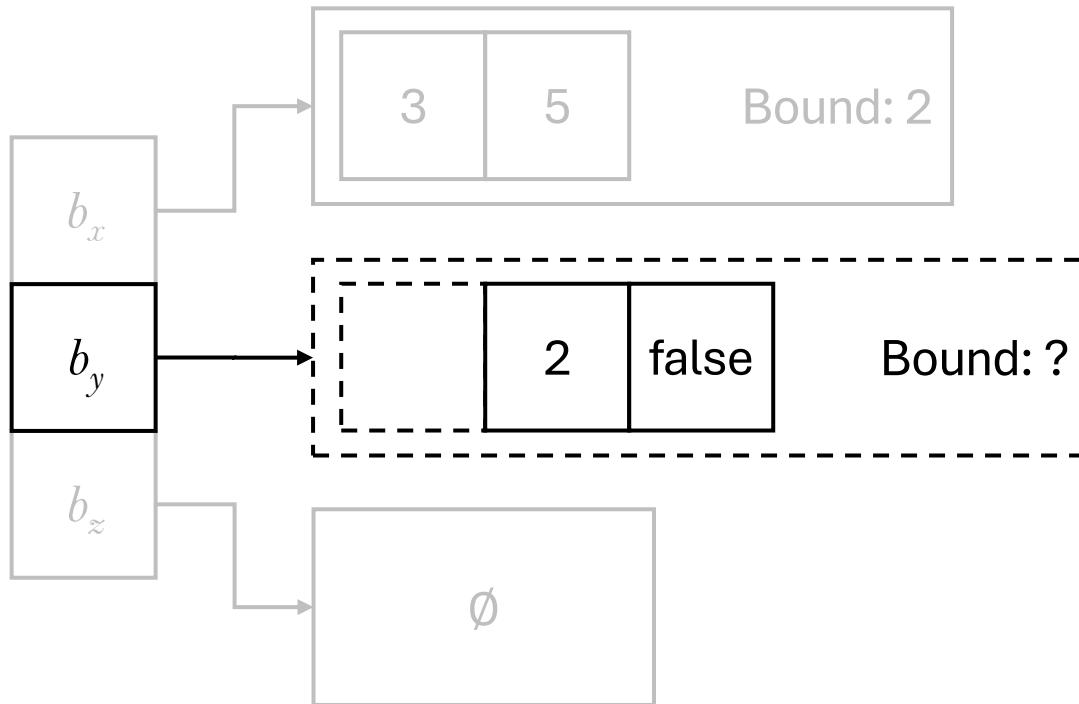
Equivalent to:

$$b_x \mapsto 3 \quad \star \quad b_x + 1 \mapsto 5 \quad \star \quad \text{Bound}(b_x; 2)$$

Block-offset Model (recap)

Resource Assertions

$$E \mapsto E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

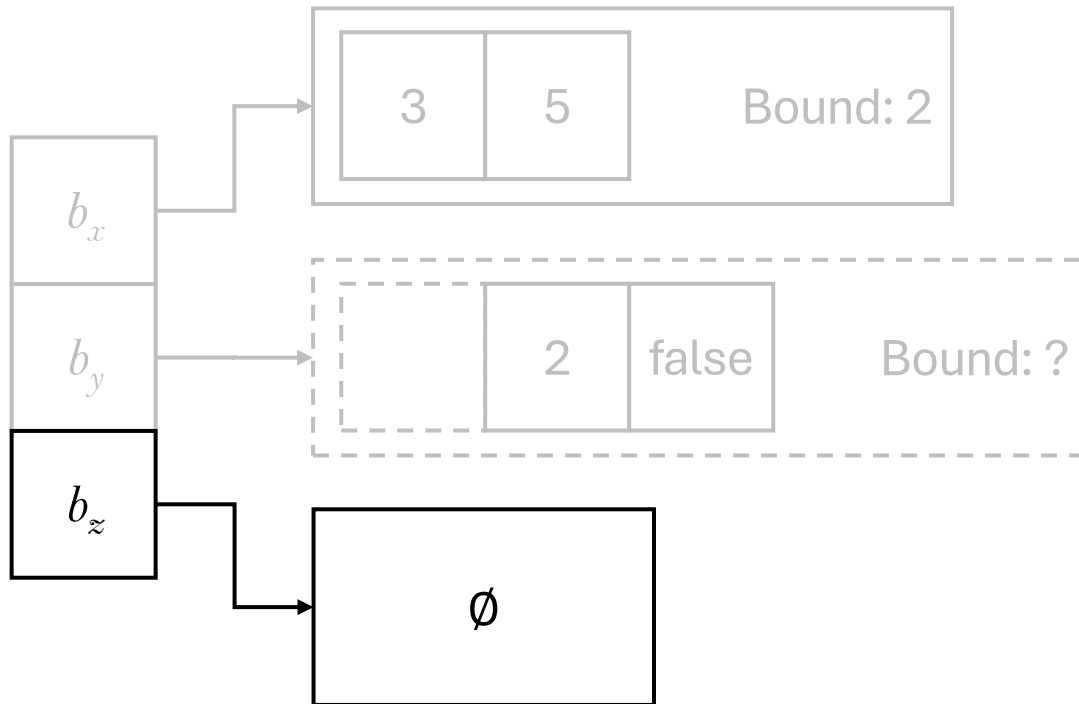


$$b_x \mapsto^b 3, 5 \quad \star \quad b_y + 1 \mapsto 2, \text{false} \quad \star \quad b_z \mapsto \emptyset$$

Block-offset Model (recap)

Resource Assertions

$$E \mapsto E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

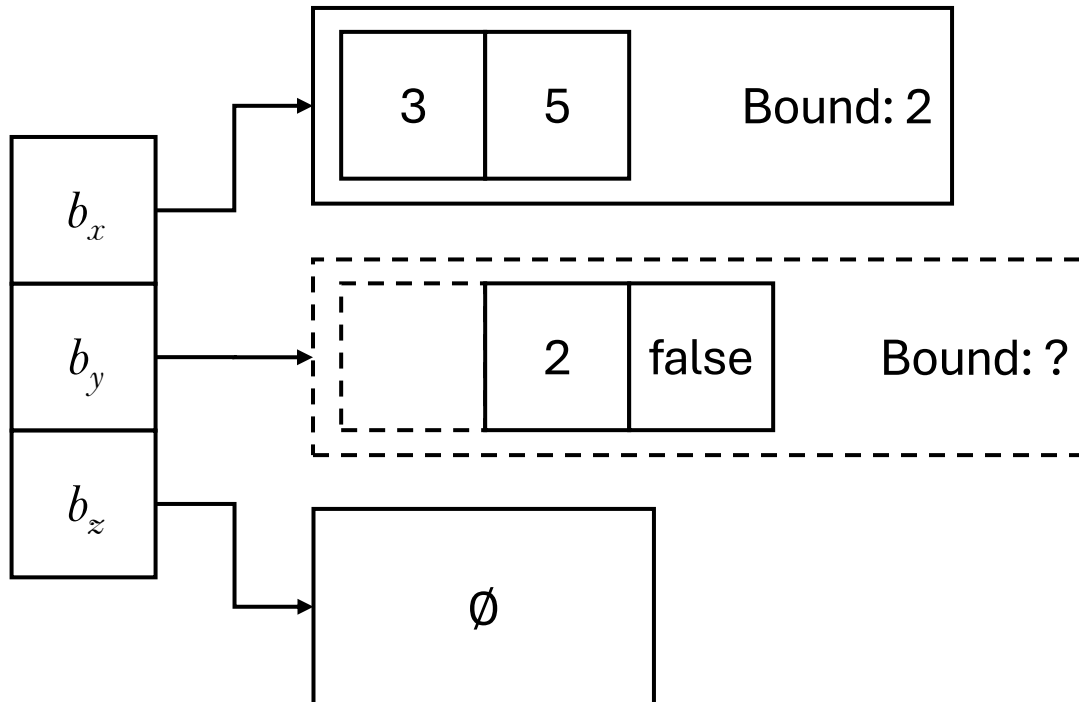


$$b_x \mapsto^b 3, 5 \quad \star \quad b_y + 1 \mapsto 2, \text{false} \quad \star \quad b_z \mapsto \emptyset$$

Extended C Block-offset Model

Partial Memory

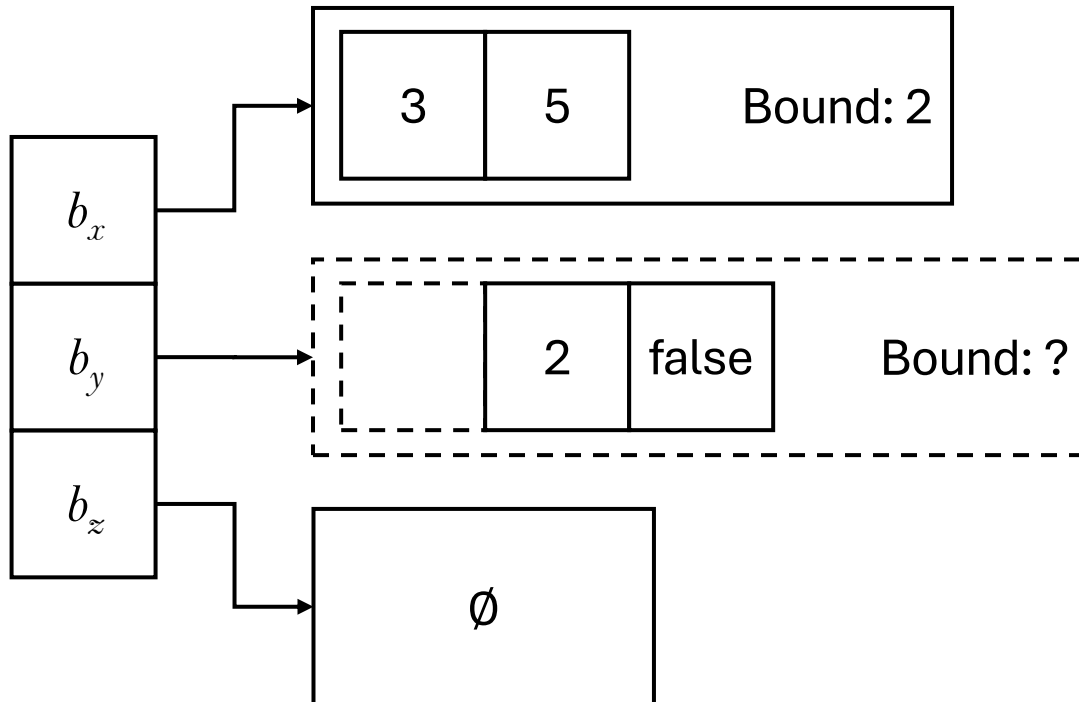
$$h : \text{Loc} \rightarrow_{\text{fin}} ((\mathbb{N} \rightarrow_{\text{fin}} \text{Val}) \times \mathbb{N}^?)$$



Extended C Block-offset Model

Partial Memory

$$h : \text{Loc} \rightarrow_{\text{fin}} ((\mathbb{N} \rightarrow_{\text{fin}} \text{Val}) \times \mathbb{N}^?)$$



C represents values in memory as sequence of bytes
– thus we ensure bytes can be stored in memory

+

We also ensure abstract pointers (b, o) can be stored in memory.

- Achieved by storing abstract pointer byte fragments of the form (b, o, i) , where i represents the i^{th} byte of the abstract pointer (b, o)

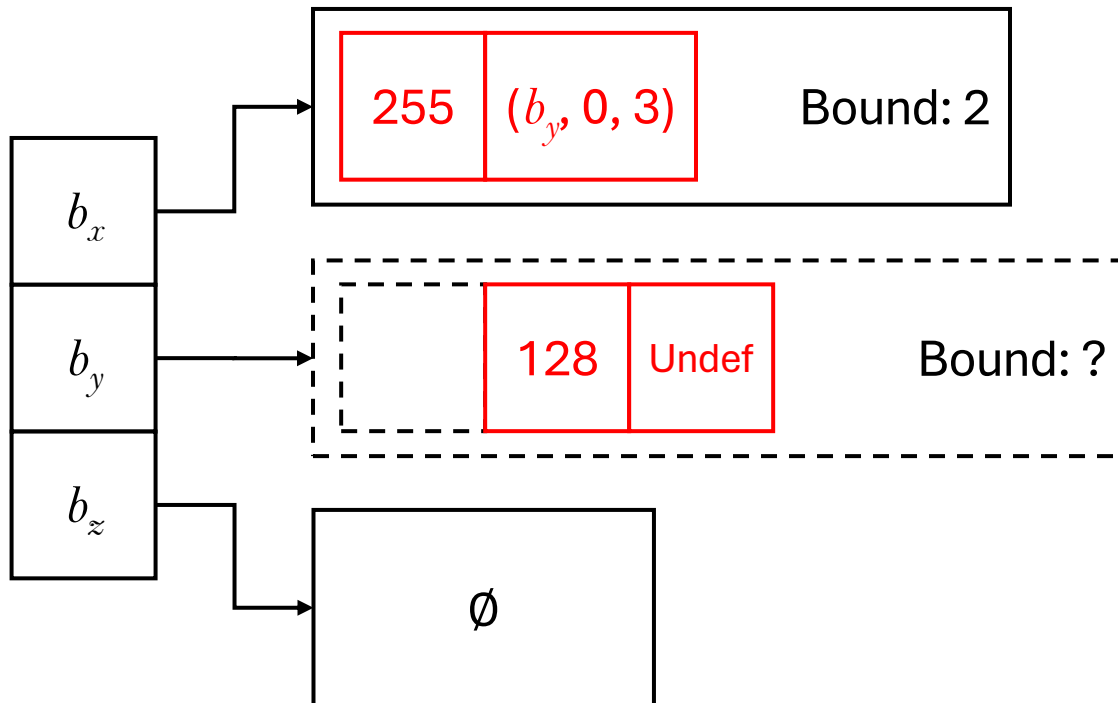
+

We also consider uninitialised values from, e.g. malloc or holes in structs

Extended C Block-offset Model

Partial Memory

$$h : \text{Loc} \rightarrow_{\text{fin}} ((\mathbb{N} \rightarrow_{\text{fin}} \text{MemVal}) \times \mathbb{N}^?)$$

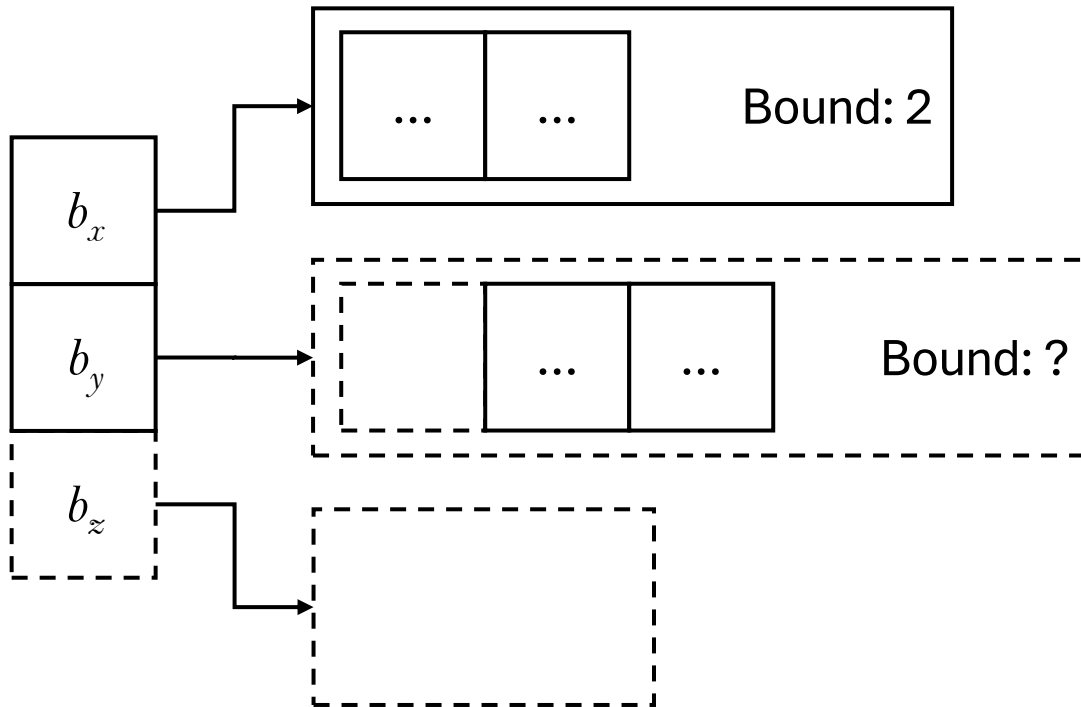


In short, we replace Val with **MemVal**, where:
MemVal := Byte n (for $0 \leq n < 256$)
 | Ptr fragment $b \ o \ i$ (for $0 \leq i < \text{size of pointer}$)
 | Undef

Extended C Block-offset Model: Alloc

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$

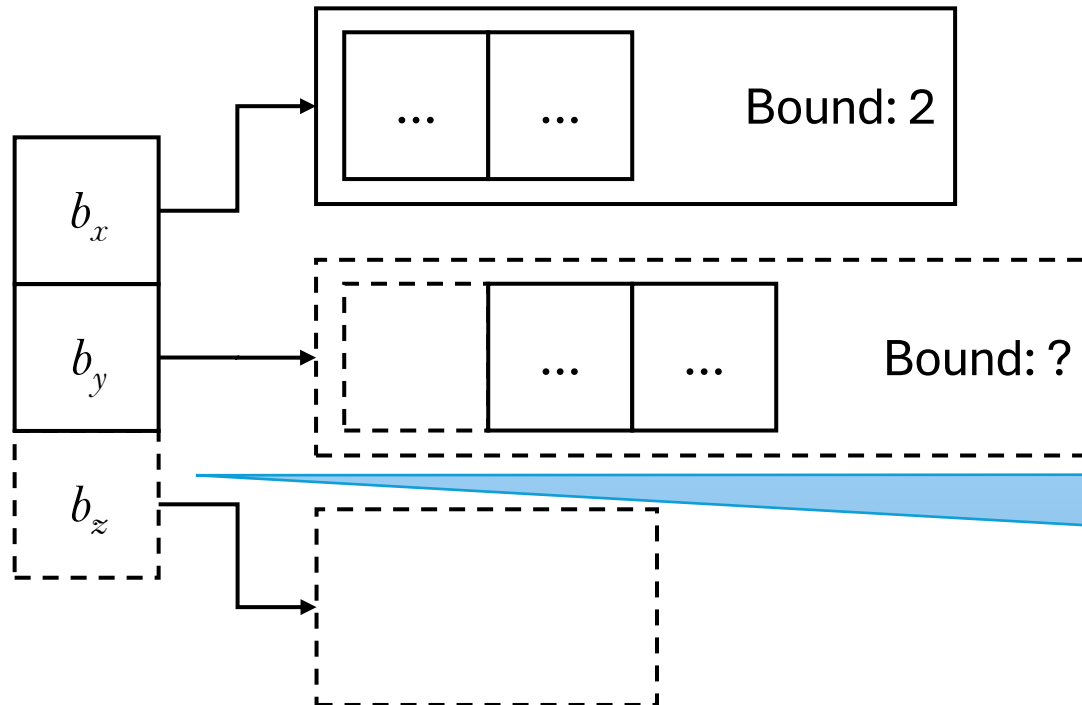


$\text{alloc}(\text{sizeof}(\text{int}))$

Extended C Block-offset Model: Alloc

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$



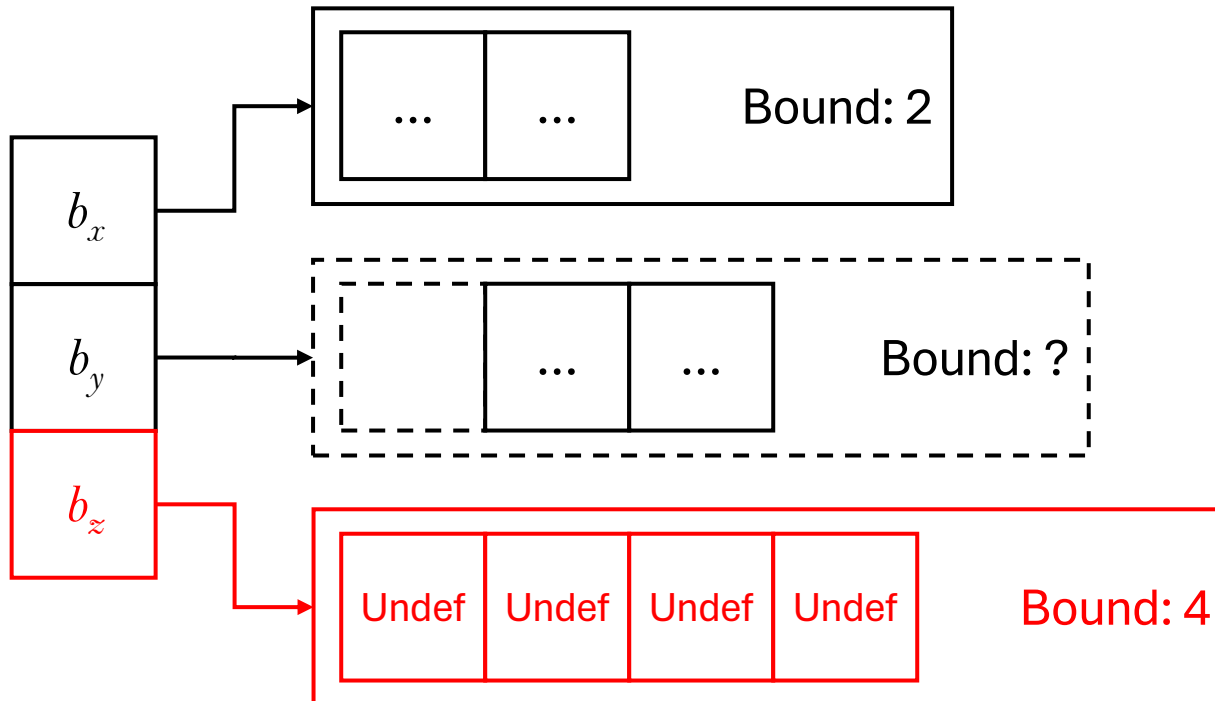
$\text{alloc}(\text{sizeof}(\text{int}))$

Same as block offset memory – uses a fresh block (in this case, b_z)

Extended C Block-offset Model: Alloc

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$

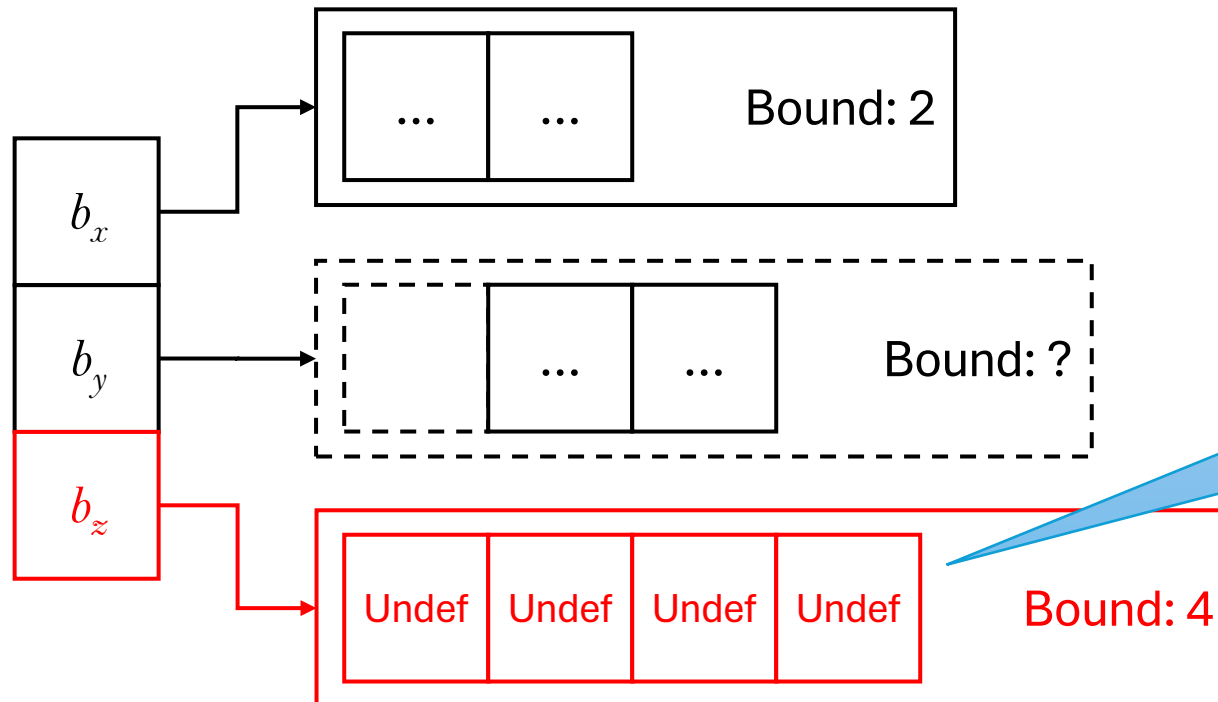


$\text{alloc}(\text{sizeof}(\text{int}))$

Extended C Block-offset Model: Alloc

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$



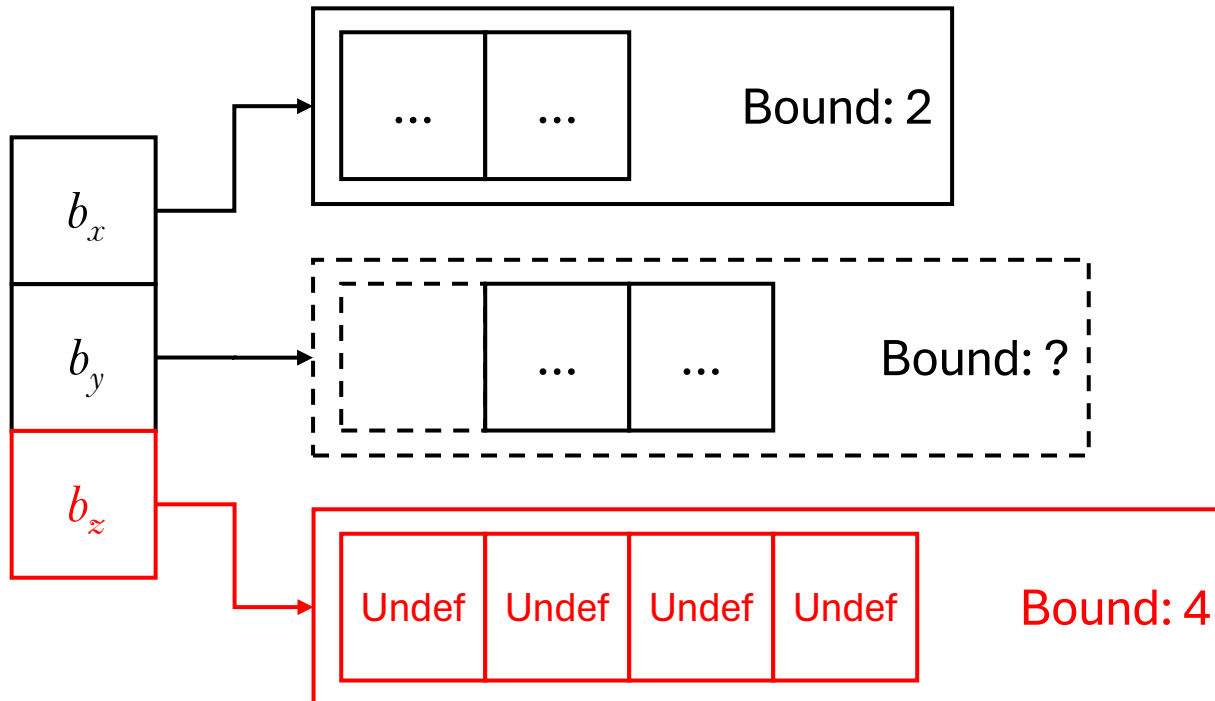
$\text{alloc}(\text{sizeof}(\text{int}))$

We state every allocated cell is initially uninitialised (as per malloc specs)

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$

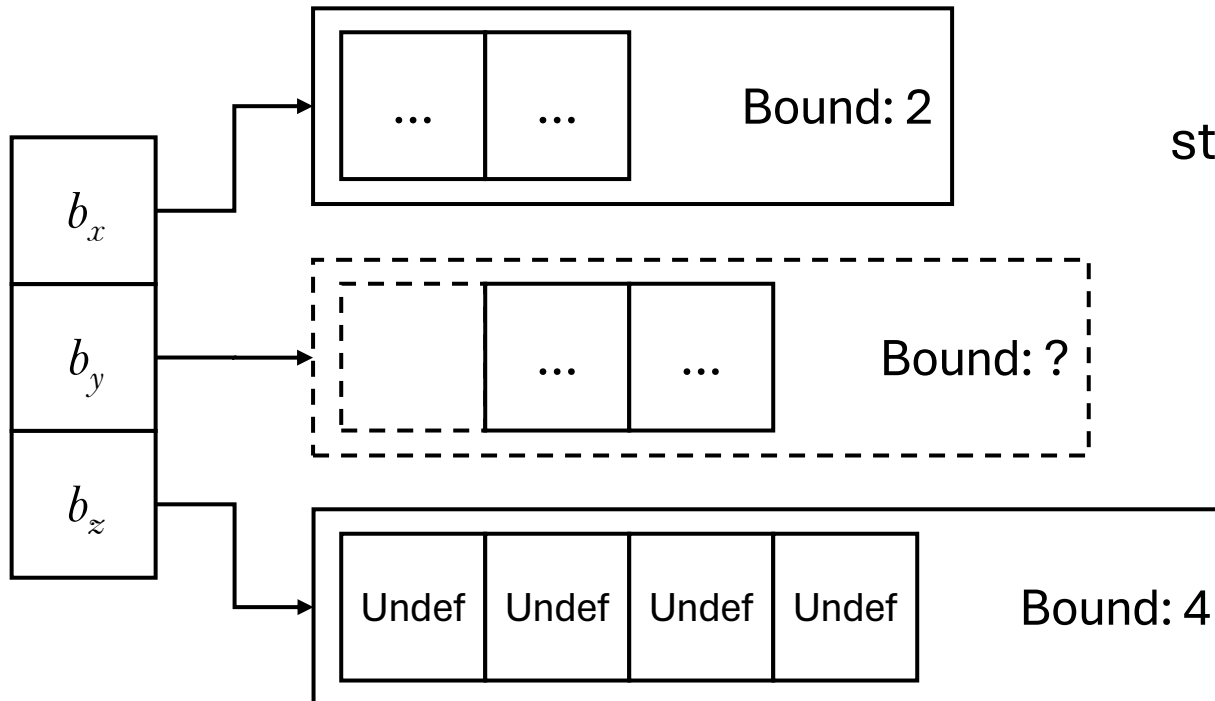


$\text{alloc}(\text{sizeof}(\text{int}))$ returns b_z

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$

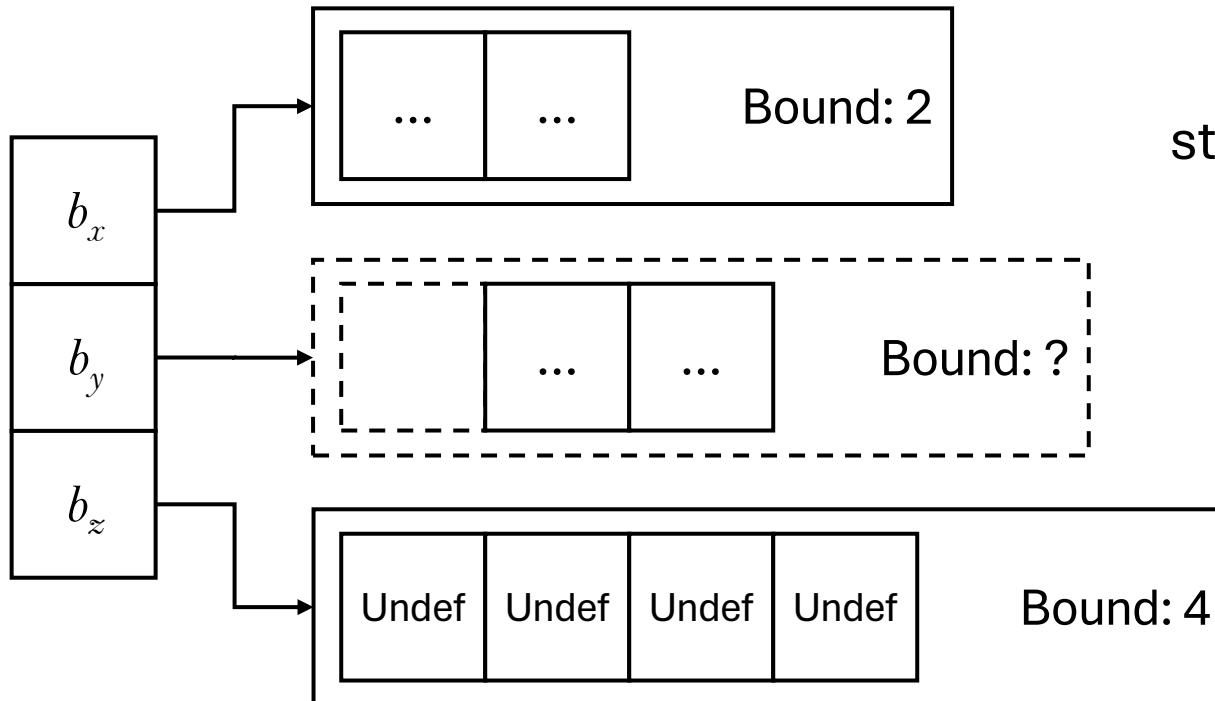


$\text{store}(\text{"unsigned int"}, b_z, 0, 2^{16} - 1)$

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\boxed{E}, E, E) \mid \text{store}(\boxed{E}, E, E, E)$



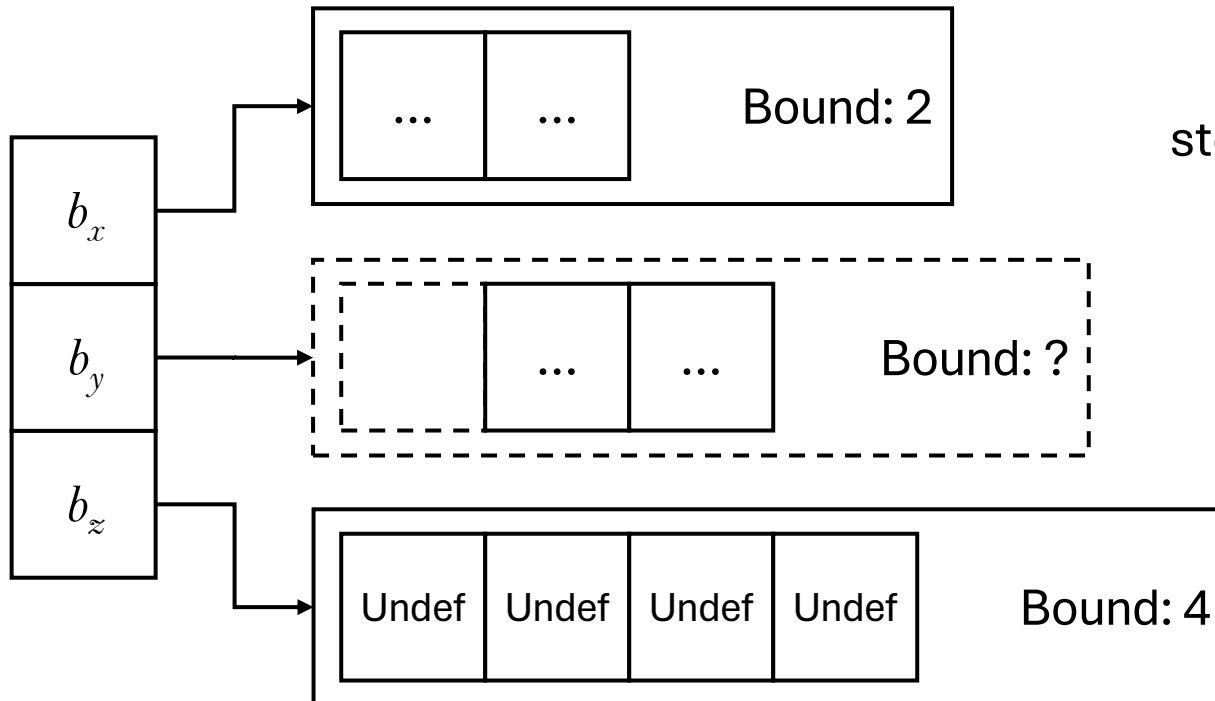
$\text{store}(\boxed{\text{"unsigned int"}}, b_z, 0, 2^{16} - 1)$

We now specify which chunk (i.e. primitive C type) we want to load/store (e.g. unsigned int)

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$

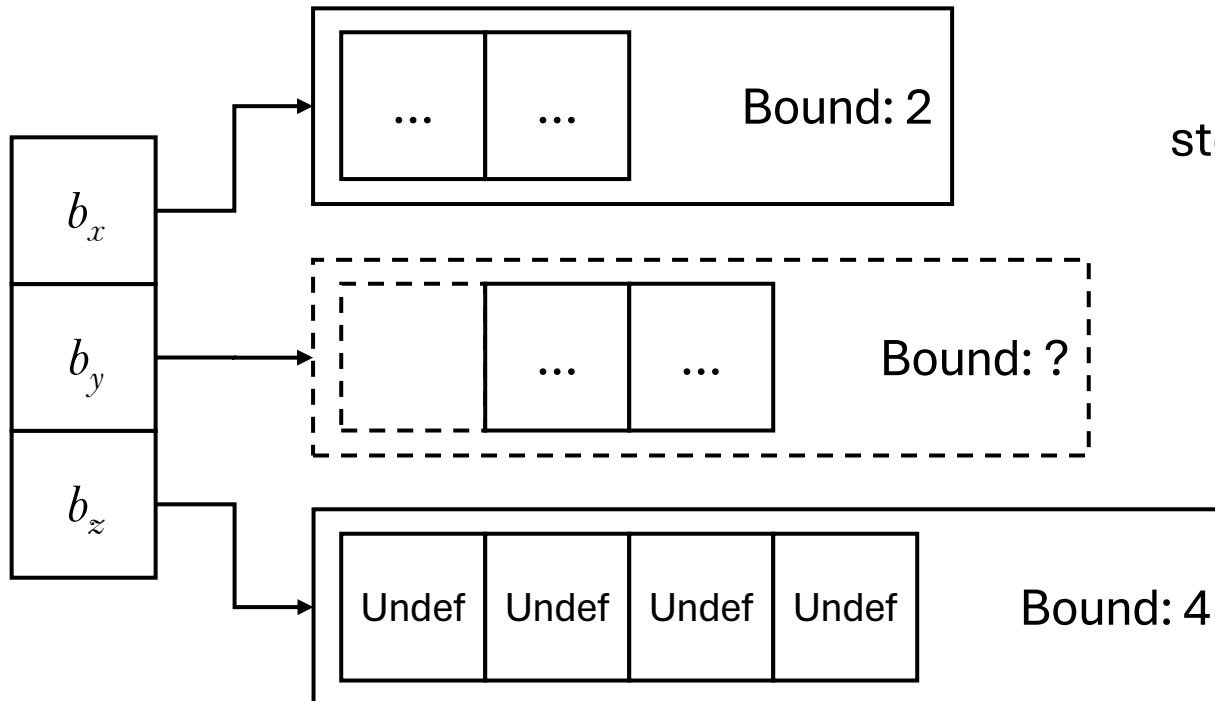


$\text{store}(\text{"unsigned short"}, b_z, 0, 2^{16} - 1)$

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$



$\text{store}(\text{"unsigned short"}, b_z, 0, 2^{16} - 1)$

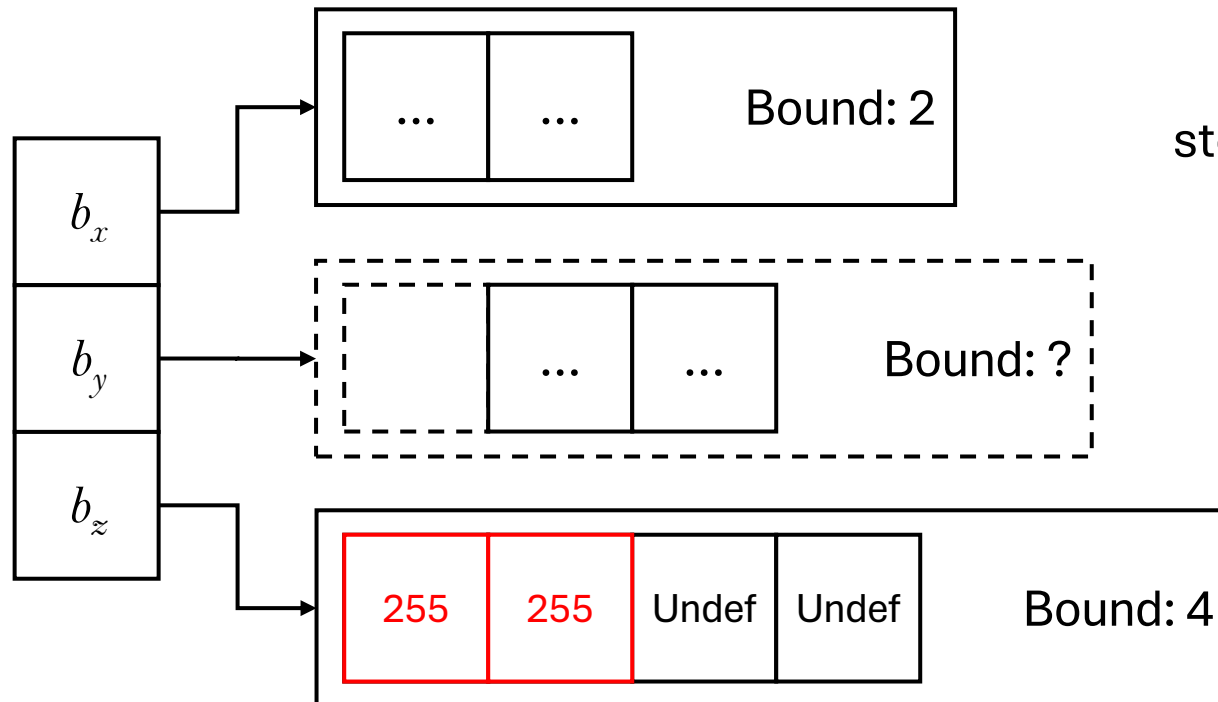
At block b_z and offset 0, store value $2^{16} - 1$

- $2^{16} - 1$ stored as an unsigned short (i.e. 2 byte value)
- encoded into sequence of 2 bytes (i.e. we have [255, 255])

Extended C Block-offset Model

Memory Actions

$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$



$\text{store}(\text{"unsigned short"}, b_z, 0, 2^{16} - 1)$

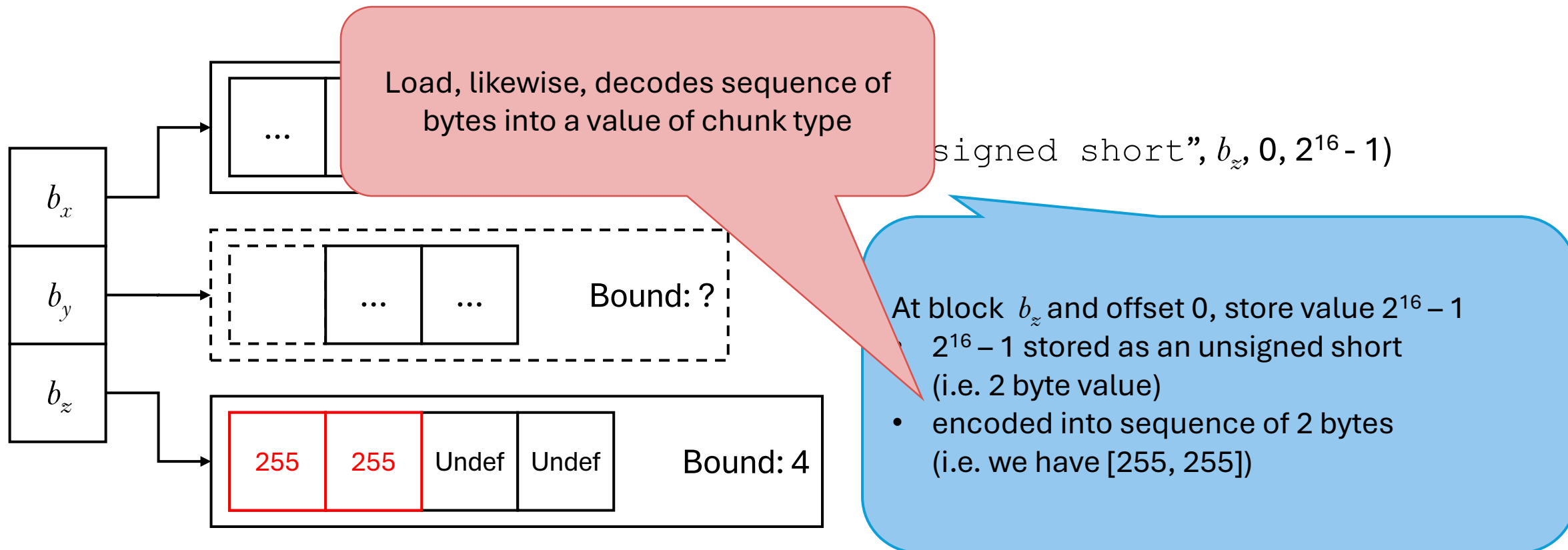
At block b_z and offset 0, store value $2^{16} - 1$

- $2^{16} - 1$ stored as an unsigned short (i.e. 2 byte value)
- encoded into sequence of 2 bytes (i.e. we have [255, 255])

Extended C Block-offset Model

Memory Actions

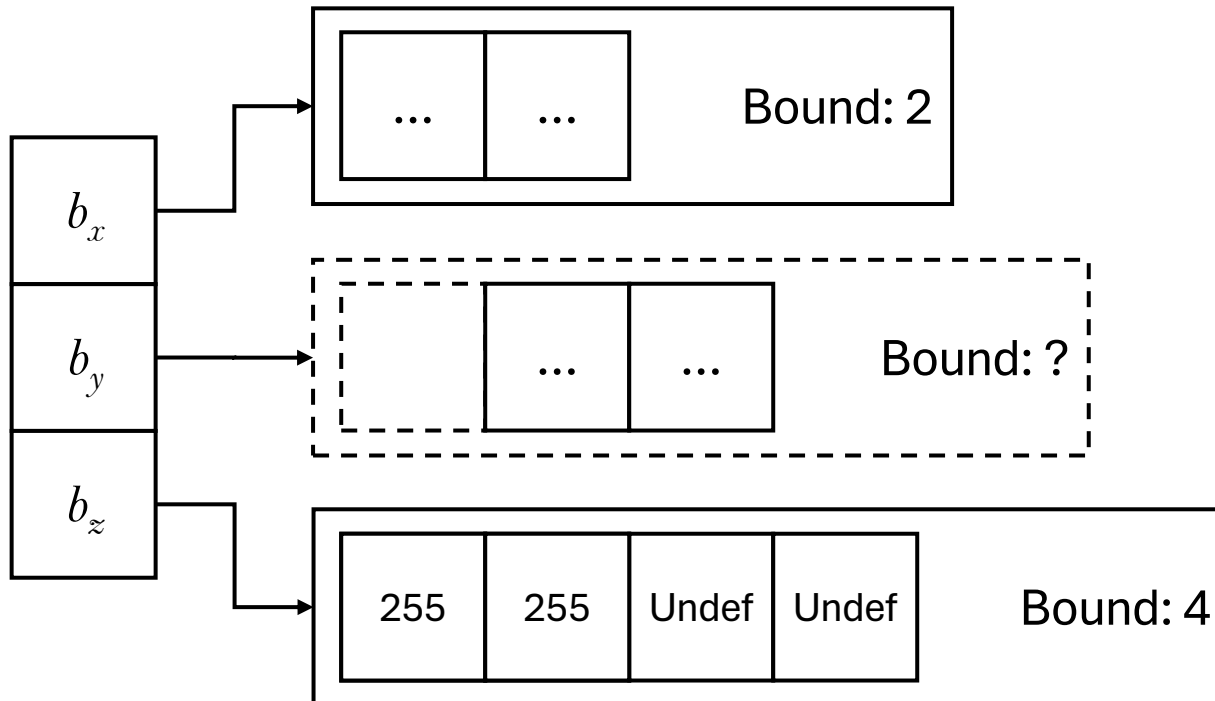
$\text{alloc}(E) \mid \text{free}(E) \mid \text{load}(\textcolor{red}{E}, E, E) \mid \text{store}(\textcolor{red}{E}, E, E, E)$



Extended C Block-offset Model

Resource Assertions

$$E \mapsto_{\tau} E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

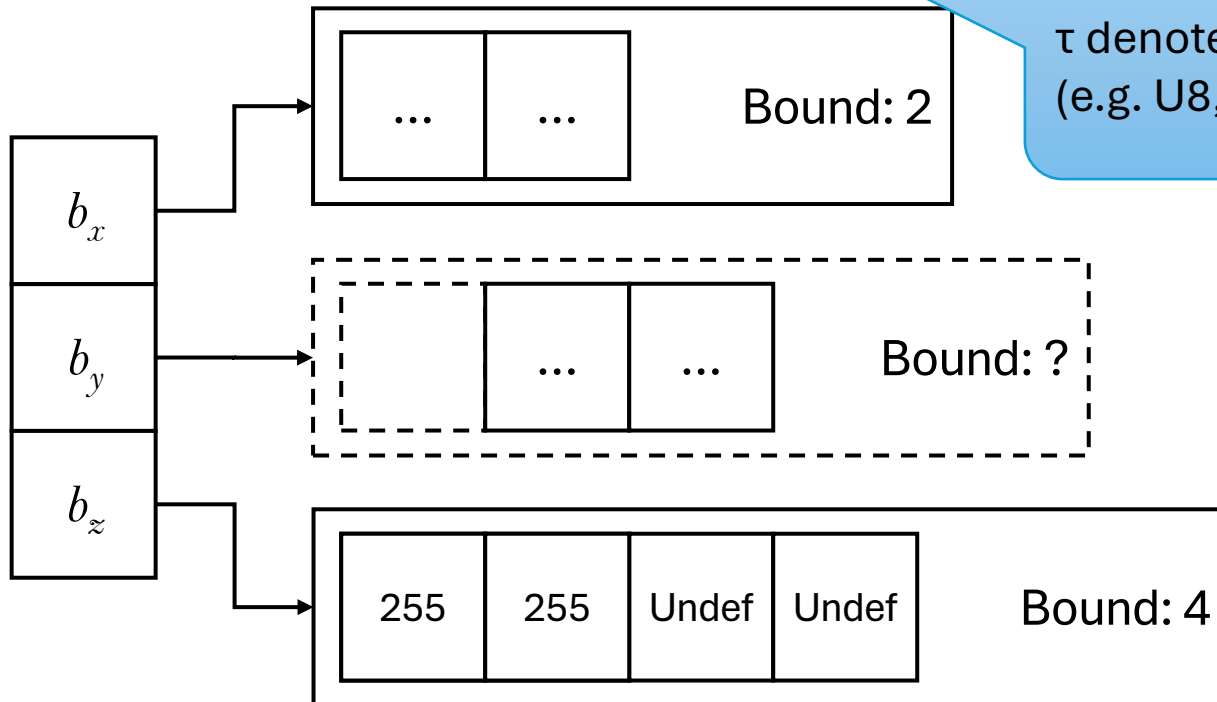


Extended C Block-offset Model

Resource Assertions

$$E \mapsto_{\tau} E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

τ denotes chunk size
(e.g. U8, U16, I32, etc.)

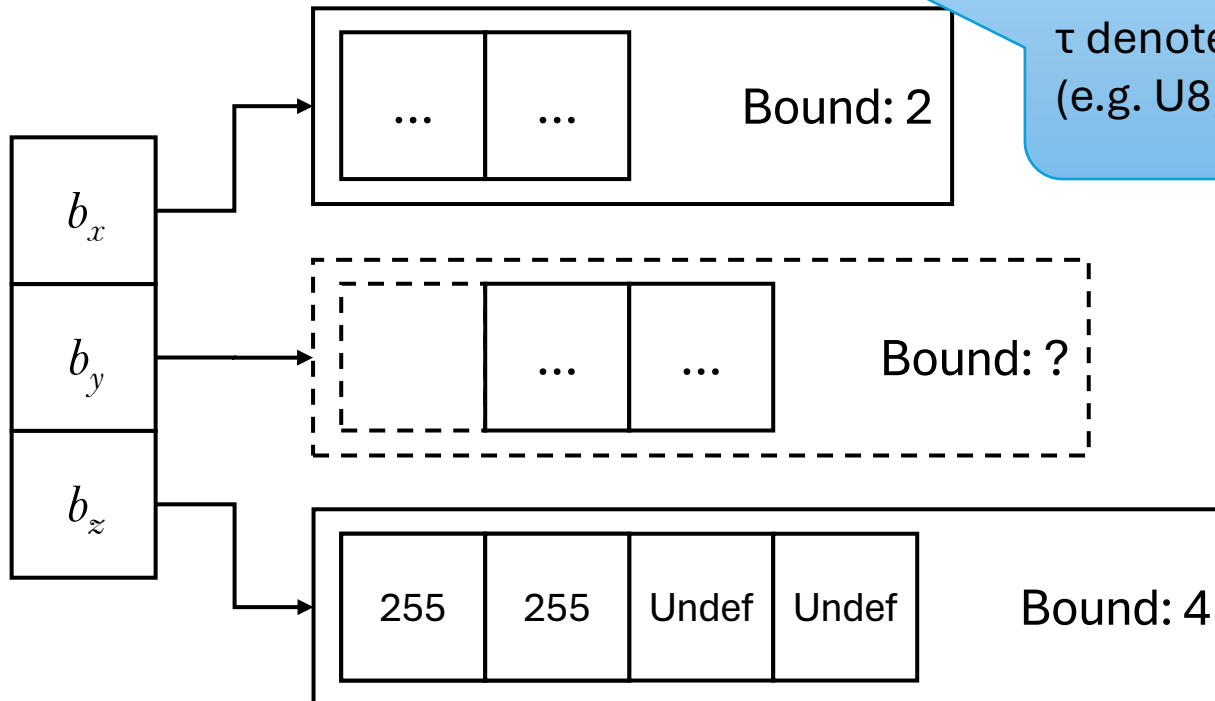


Extended C Block-offset Model

Resource Assertions

$$E \mapsto_{\tau} E \mid \text{bound}(E; E) \mid E \mapsto \emptyset$$

τ denotes chunk size
(e.g. U8, U16, I32, etc.)



b_z	$\mapsto_{U16}$	$2^{16} - 1$	★
$b_z + 2$	$\mapsto_{U8}$	Undef	★
$b_z + 3$	$\mapsto_{U8}$	Undef	★

Bound(b_z ; 4)

Extended C Block-offset Model

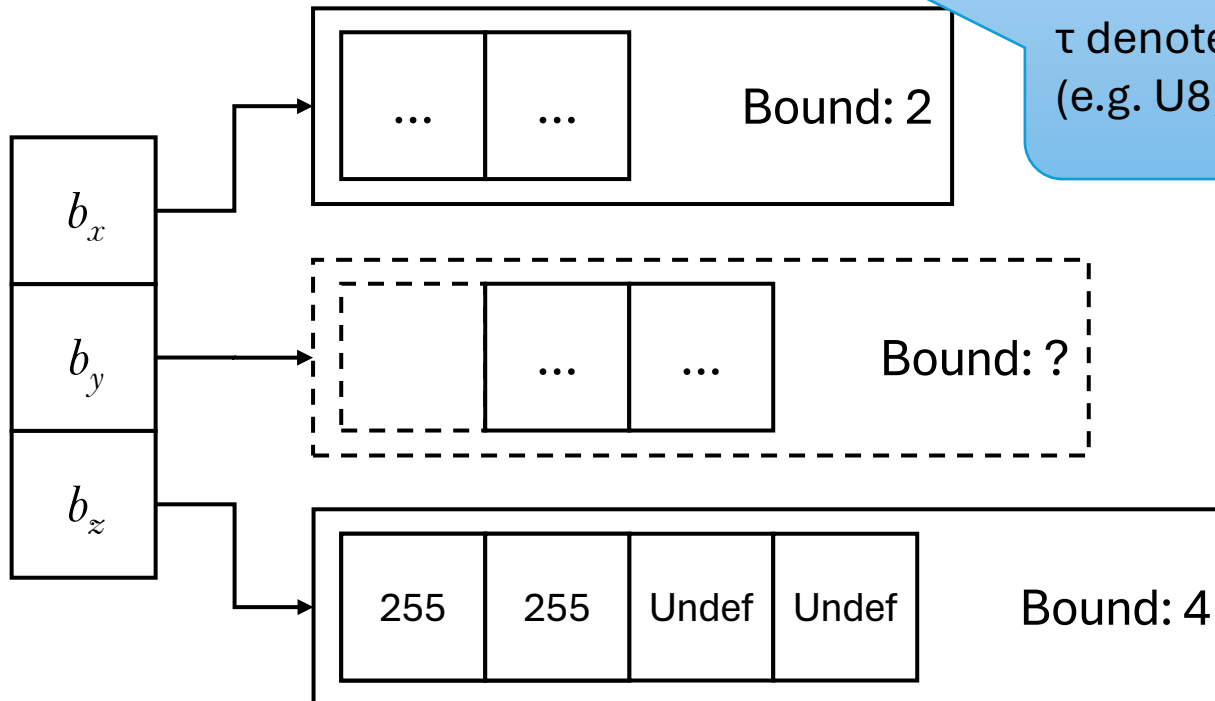
Resource Assertions

$E \mapsto_{\tau} E \mid \text{bound}$

Alternate way to express the same assertion
(avoiding mentioning the Bound predicate):

$b_z \mapsto_{U8}^b 255, 255, \text{Undef}, \text{Undef}$

τ denotes chunk size
(e.g. U8, U16, I32, etc.)



$b_z \mapsto_{U16} 2^{16} - 1 \quad \star$
 $b_z + 2 \mapsto_{U8} \text{Undef} \quad \star$
 $b_z + 3 \mapsto_{U8} \text{Undef} \quad \star$
 $\text{Bound}(b_z; 4)$

Dealing with holes in C structs

Consider the following C struct:

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```


Dealing with holes in C structs

Consider the following C struct:

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

Represented in memory as:



Dealing with holes in C structs

Consider the following C struct:

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

Represented in memory as:



Holes exist due to
struct alignment

Dealing with holes in C structs

Consider the following C struct:

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

For every struct in C, Gillian-C generates a corresponding user-defined predicate

Represented in memory as:



Holes exist due to
struct alignment

Dealing with holes in C structs

Consider the following C struct:

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

Represented in memory as:



Holes exist due to
struct alignment

For every struct in C, Gillian-C generates a corresponding user-defined predicate

```
ln_struct(b, o; data, next) {  
    is_int(data) ★  
    (b+o)  $\mapsto_{i32}$  data ★  
    (b+o+4)  $\mapsto_{u32}$  Undef ★  
    is_ptr(next) ★  
    (b+o+8)  $\mapsto_{ptr}$  next  
}
```

Using user-defined struct predicates in C

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

```
ln_struct(b, o; data, next) {  
    is_int(data) ★  
    (b+o) ↦i32 data ★  
    (b+o+4) ↦u32 Undef ★  
    is_ptr(next) ★  
    (b+o+8) ↦ptr next  
}
```

In Gillian-C, the `sll` user-defined predicate is defined as follows:

```
/*@ pred sll(+p, alpha) {  
    p -m> struct ln { #head; #tail } *  
    alpha == #head::#beta *  
    sll(#tail, #beta) *  
    is_size_t(len alpha);  
    p == NULL * alpha == []  
*/
```

Using user-defined struct predicates in C

```
typedef struct ln {  
    int data;  
    struct ln *next;  
} SLL;
```

Here,

`p -m> struct ln { #head; tail }`
...is equivalent to...
`ln_struct(p.1, p.2; #head, tail)`

```
ln_struct(b, o; data, next) {  
    is_int(data) ★  
    (b+o) ↦i32 data ★  
    (b+o+4) ↦u32 Undef ★  
    is_ptr(next) ★  
    (b+o+8) ↦ptr next  
}
```

In Git, the SLL user-defined predicate is defined as follows:

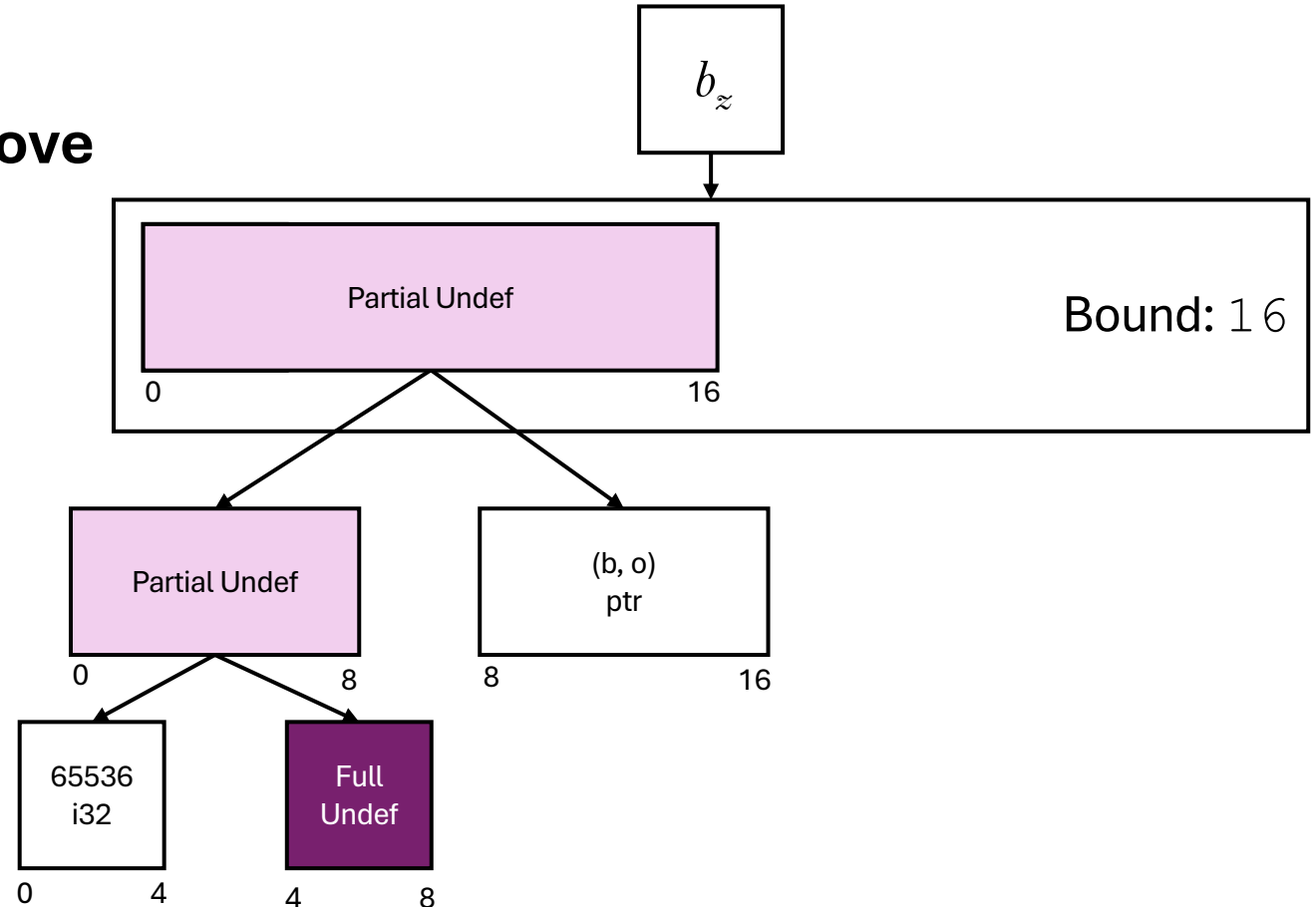
```
/*@ pred sll(+p, alpha) {  
    p -m> struct ln { #head; #tail } *  
    alpha == #head::#beta *  
    sll(#tail, #beta) *  
    is_size_t(len alpha);  
    p == NULL * alpha == []  
*/
```

Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o)  $\mapsto_{i32}$  data ★  
  (b+o+4)  $\mapsto_{u32}$  Undef ★  
  is_ptr(next) ★  
  (b+o+8)  $\mapsto_{ptr}$  next  
}
```

load("int", b_z , 0) returns ?

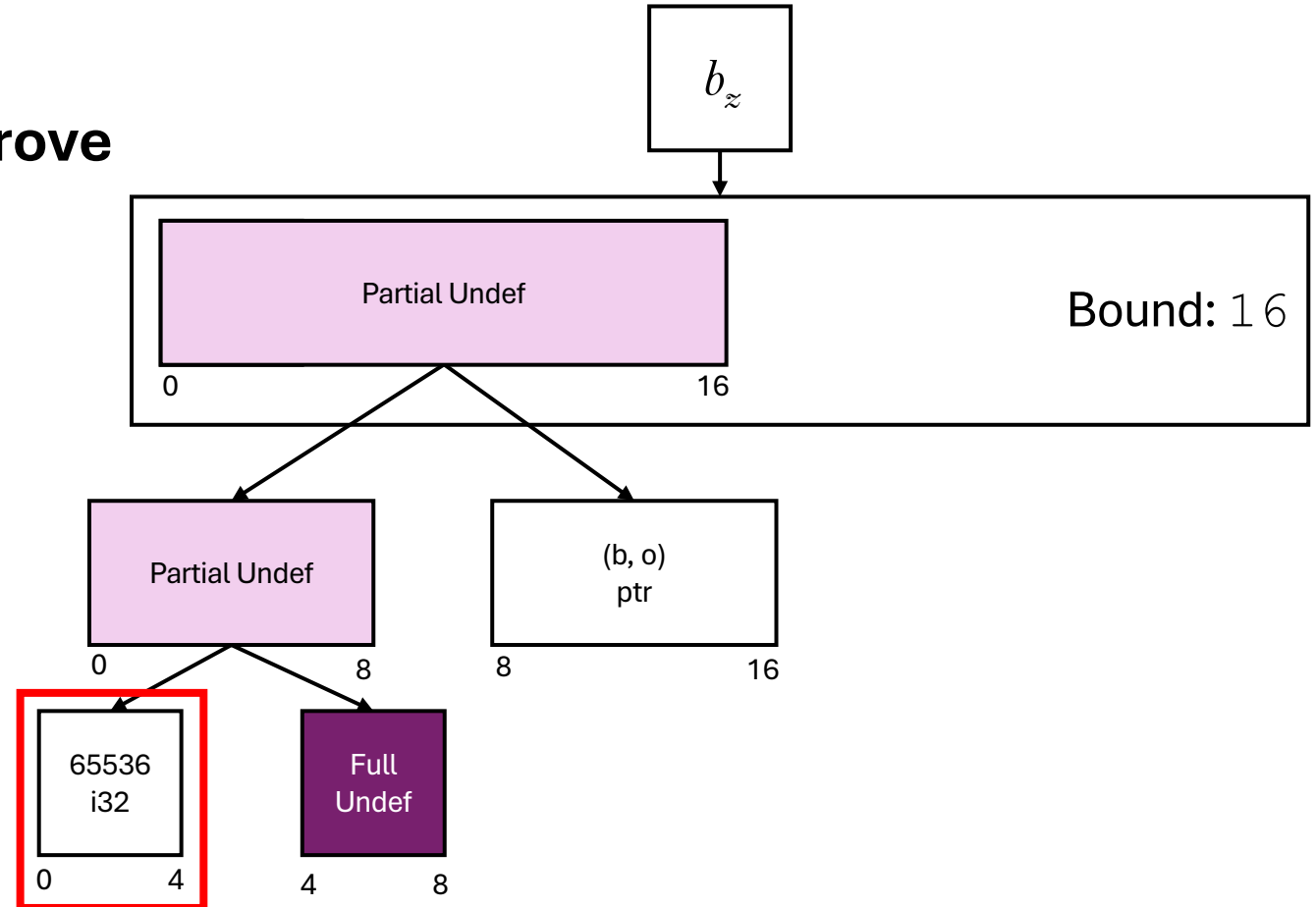


Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o)  $\mapsto_{i32}$  data ★  
  (b+o+4)  $\mapsto_{u32}$  Undef ★  
  is_ptr(next) ★  
  (b+o+8)  $\mapsto_{ptr}$  next  
}
```

load("int", b_z , 0) returns 65535

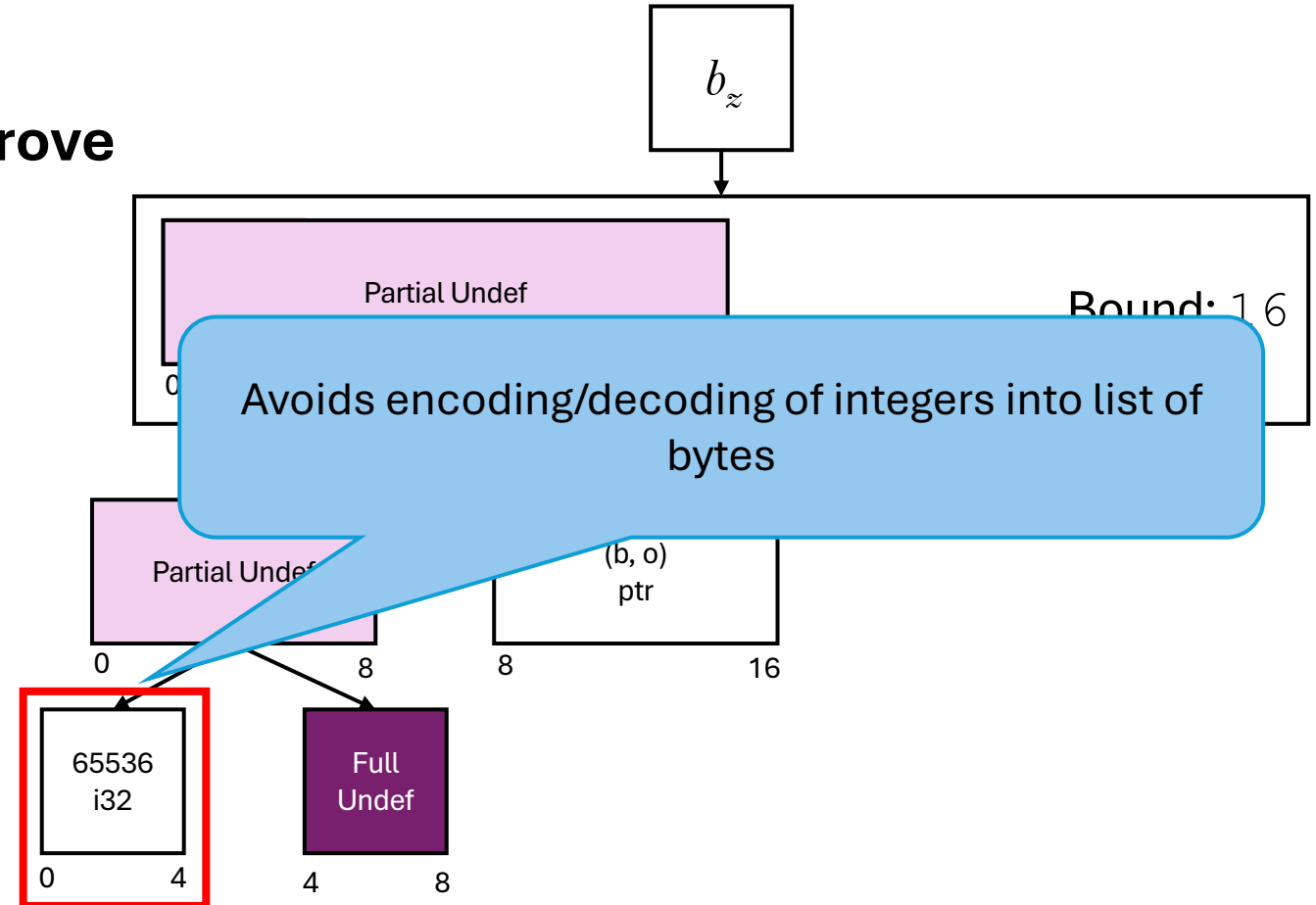


Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o)  $\mapsto_{i32}$  data ★  
  (b+o+4)  $\mapsto_{u32}$  Undef ★  
  is_ptr(next) ★  
  (b+o+8)  $\mapsto_{ptr}$  next  
}
```

load("int", b_z , 0) returns 65535

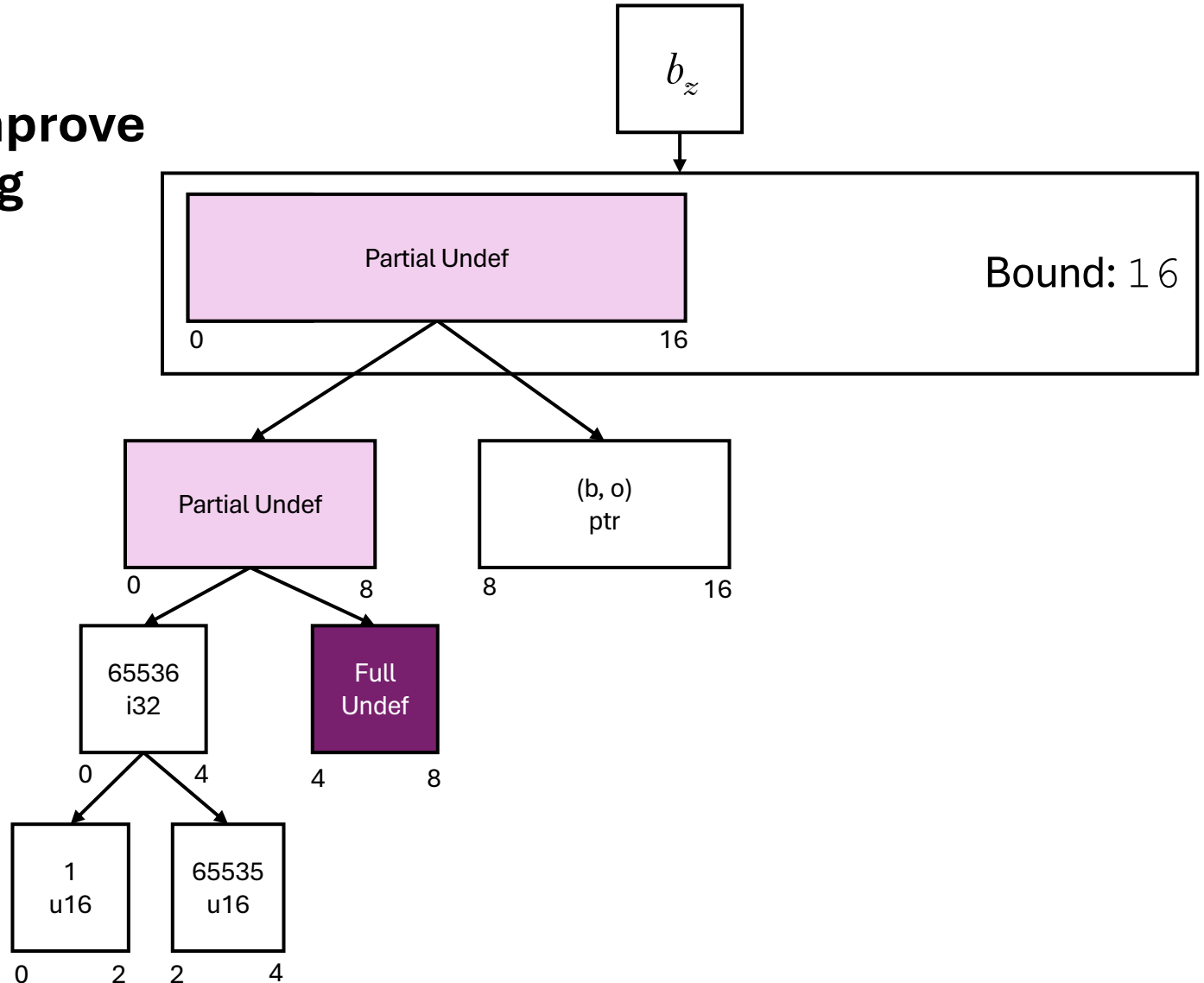


Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o)  $\mapsto_{i32}$  data ★  
  (b+o+4)  $\mapsto_{u32}$  Undef ★  
  is_ptr(next) ★  
  (b+o+8)  $\mapsto_{ptr}$  next  
}
```

load("unsigned short", b_z , 2) returns ?

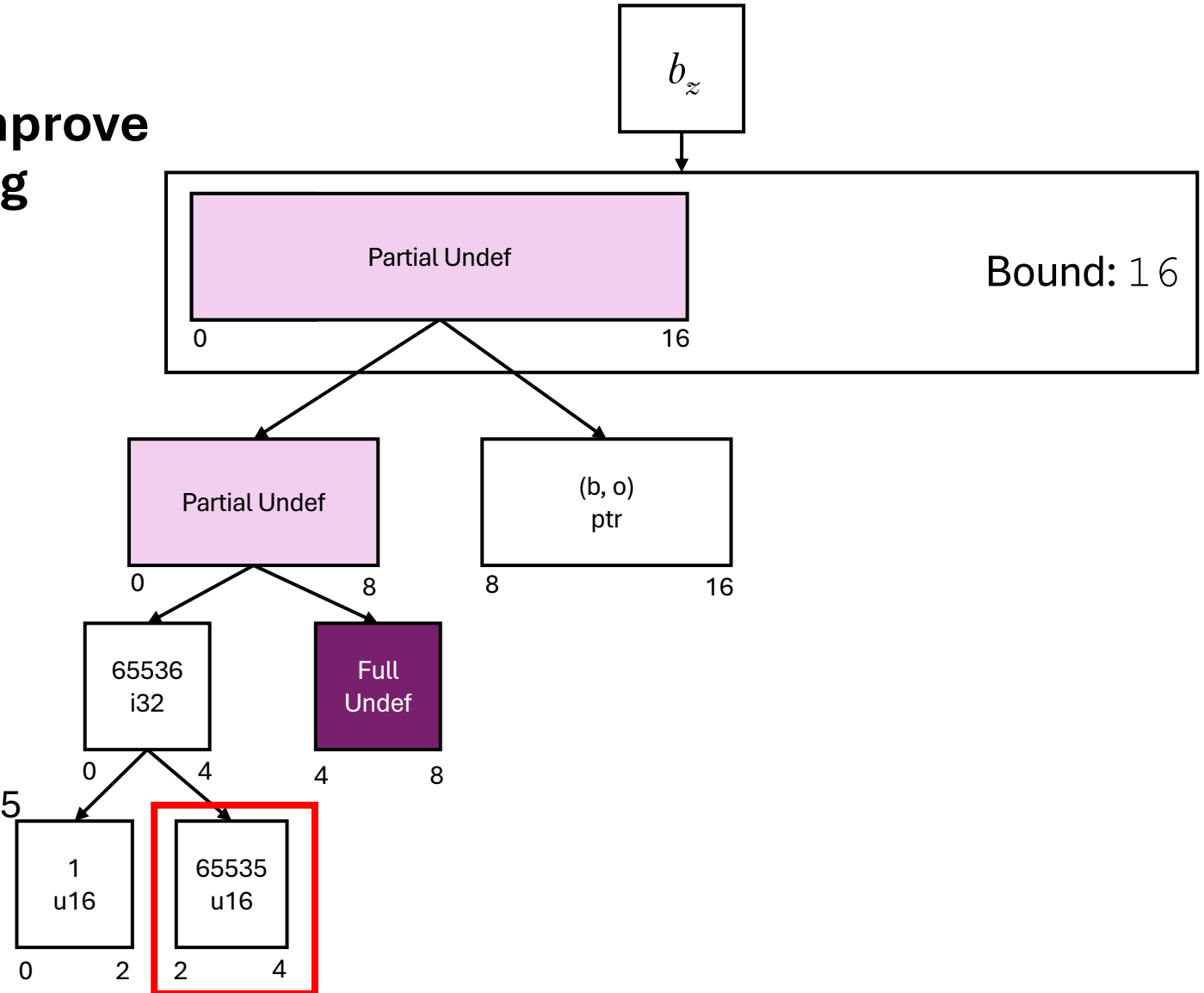


Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o) ↦i32 data ★  
  (b+o+4) ↦u32 Undef ★  
  is_ptr(next) ★  
  (b+o+8) ↦ptr next  
}
```

load("unsigned short", b_z , 2) returns 65535

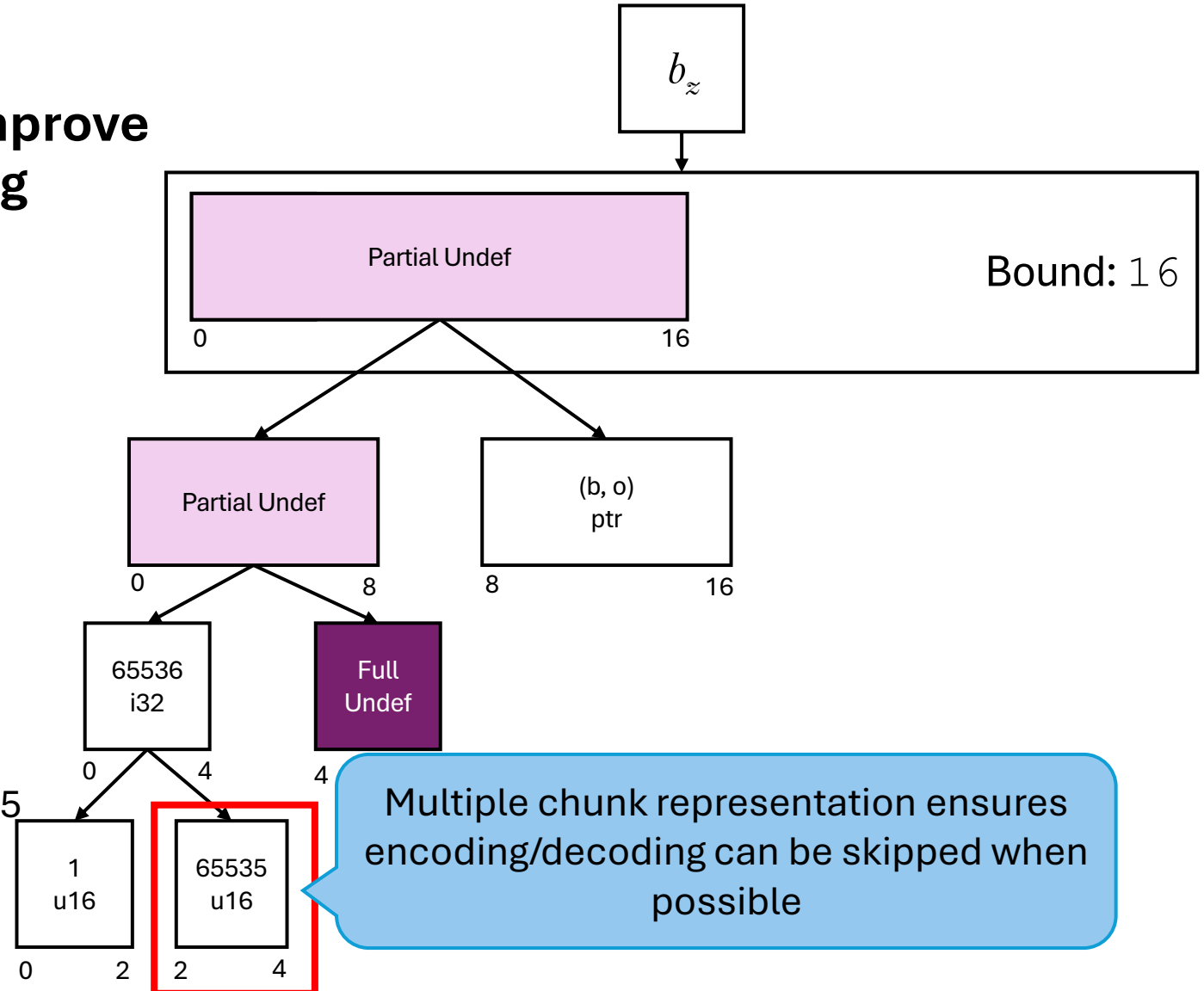


Further optimisations

Uses block-tree structure to improve efficiency of loading and storing

```
ln_struct(b, o; data, next) {  
  is_int(data) ★  
  (b+o)  $\mapsto_{i32}$  data ★  
  (b+o+4)  $\mapsto_{u32}$  Undef ★  
  is_ptr(next) ★  
  (b+o+8)  $\mapsto_{ptr}$  next  
}
```

load("unsigned short", b_z , 2) returns 65535



C demo

Gillian-C with heap tree memory

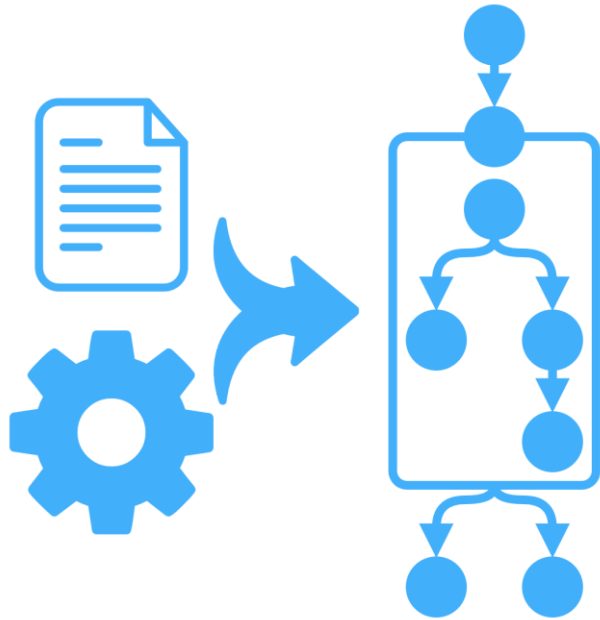
<https://youtu.be/zsJLskHjmG0>



Nat Karmios

Further debugging work

Principled connection between
source semantics & analysis for
source-level debugging

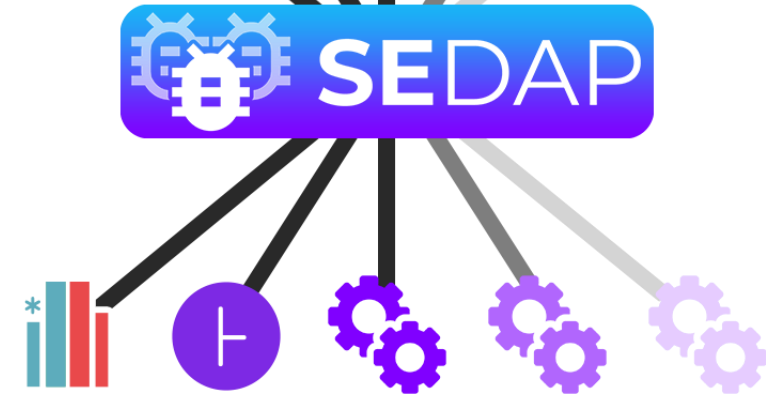


General infrastructure for
debugging CSE-based tools

Editors / IDEs



Symbolic tools



OPLSS Summer School

Lecture 1: Separation Logic

- An introduction to Separation Logic, a modern Hoare Logic
- The specification and verification of heap-manipulating programs
- Tools for verification and true bug detection, based on compositional symbolic execution

Lecture 2: From Separation Logic to Compositional Symbolic Execution

- Experience with Separation Logic
- Core Compositional Symbolic Execution
- Automatic whole-program symbolic analysis and bug detection

Lecture 3: Compositional Symbolic Execution

- Compositional symbolic execution, parametric on the state
- Semi-automatic verification of function specifications

Lecture 4: Semi-automatic Verification and Automatic Bug Detection

- An introduction to the Gillian platform
- Gillian-While with simple block-offset memory and Gillian-C with full-scale C memory

Future Work

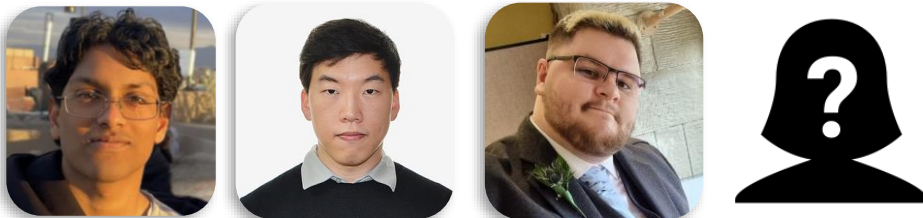
- Next-generation Compositional Symbolic Execution



- Coq Formalism and Verification



- Lillian (Next-generation Gillian)



The CSE and Gillian Team: Past and Current Members



Philippa Gardner



Diego Cupello



Andreas Lööw



Simon Park



Sacha-Élie Ayoun



Caroline Cronjäger



Petar Maksimović



José Frogado Santos



Nat Karmios



Daniele Nantes



Opale Sjöstedt



Shiva Tamil Kumaran